Flaris Virtual Machine — Bytecode Specification

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.6.0 **Generated** 2026-10-24 **License** see repository

© 2026 SolidInvention AB — created and maintained by SolidInvention AB

This document is the normative reference for implementers of independent Flaris compilers, loaders, disassemblers, or virtual machines. It describes the on-disk bytecode format and the observable execution semantics of every instruction. The companion diagram flx-format.svg illustrates the container layout; where diagram and text disagree, this text is authoritative.

---

1. Introduction

1.1 Scope

The specification covers:

- the `.flx` binary container: header, signature, string pool, chunks, bundles
- the serialization of constants, functions and classes
- the instruction encoding and the semantics of all 203 opcodes
- the abstract machine: value system, operand stack, call frames, fibers, exception handling
- the validation rules a conforming loader must apply before execution

It does **not** cover: the Flaris source language (see guide.md and reference.md), the built-in function library (see reference.md), the FFI plugin ABI (see ffi.md), or the optional JIT IR side-channel (a conforming VM may ignore it; see §3.7).

1.2 Conformance terminology

- **MUST / MUST NOT** — required for conformance. A loader that skips a MUST validation rule accepts malformed bytecode and is non-conforming.
- **SHOULD** — recommended; deviation must not change observable program behavior.
- **implementation note** — describes the reference implementation (`flarisvm`); an independent implementation may differ as long as observable semantics match.

1.3 Versioning

Constant	Value	Meaning
`SYS_VERSION` / `CHUNK_VERSION`	`0x01000600` (1.0.6.0)	version written into produced chunks
`MIN_SUPPORTED_BYTECODE_VERSION`	`0x01000600` (1.0.6.0)	oldest chunk version a loader MUST accept
`DEFAULT_LIB_VERSION`	`0x01000000`	default module version when unspecified

Version words pack four bytes `major.minor.patch.revision`, most-significant byte first within the 32-bit value (e.g. `0x01000008` = 1.0.0.8).

Opcode numbering is dense (Appendix A): inserting or removing an instruction renumbers every following opcode, and `MIN_SUPPORTED_BYTECODE_VERSION` is bumped in the same change. New opcodes **appended before `OP_LAST`** keep all prior numbers stable and only bump `SYS_VERSION` (older VMs reject the newer files via the version ceiling; 1.0.0.9 appended `OP_CONCAT_N` and `OP_NIL_LOCAL`, 1.0.2.0 appended `OP_CALL_WITH_THIS` and `OP_OBJ_ARITH_L`, all this way; 1.0.3.0 inserted `OP_USHR` after `OP_SHR` and raised the floor to 1.0.3.0; 1.0.4.0 added the optional import-table chunk section, §5.5, and removed `OP_DBG_FUNC_NAME`, raising the floor to 1.0.4.0; 1.0.5.0 made the `OP_ITER_NEXT` and `OP_JUMP_TABLE` targets relative, §6.6, raising the floor to 1.0.5.0; 1.0.6.0 removed `OP_ONE`, `OP_NEG_ONE` and the block forms `OP_GET_BLK_LC`, `OP_SET_BLK_LC`, `OP_SET_BLK_LLC`, `OP_LOCAL_BLK_LC` and `OP_LOCAL_BLK_LL`, the `this.a[i]` forms `OP_GET_THIS_ARR_L`, `OP_SET_THIS_ARR_L`, `OP_SET_THIS_ARR_LL` and their slot forms, and the block compound assignments `OP_ARITH_BLK_LC`, `OP_ARITH_BLK_LL`, `OP_ARITH_BLK_IC` and `OP_ARITH_BLK_IL`, and split `OP_FOREACH` into `OP_FOREACH_BEGIN` and `OP_FOREACH_NEXT`, raising the floor to 1.0.6.0). A loader MUST reject a chunk whose version is below the floor or above its own `SYS_VERSION`.

---

2. Notation and conventions

- **Byte order.** All multi-byte integers in the container *and* in instruction operands are **little-endian**, without exception.
- **Types.** `u8` / `u16` / `u32` / `u64` are unsigned little-endian integers of that width; `i8` / `i16` / `i32` / `i64` are two's-complement signed; `f32` / `f64` are IEEE 754 binary32/binary64.
- **Instruction syntax.** An instruction is written `MNEMONIC <operand:type> <operand:type> …`. The opcode itself is always one byte. Operands follow immediately, in the order listed.
- **Stack notation.** Stack effects are written `…, a, b → …, r`: the right end is the top of stack (TOS). `→ …` alone means the instruction pushes nothing.
- **`local[n]`** is slot *n* of the current call frame. **`TOS`** is the top of the operand stack. **`ip`** is the byte offset of the *next* instruction unless stated otherwise.
- Hash values are 64-bit **xxHash64 with seed 0** over the identifier's UTF-8 bytes unless stated otherwise (§4.6).

---

3. The abstract machine

3.1 Overview

The Flaris VM is a **stack machine** executing one **fiber** at a time on a single OS thread. A fiber owns:

- an operand stack of `Object*` values (initial `DEFAULT_STACK_SIZE` = 1024 slots, hard cap `MAX_STACK` = 65535),
- a call-frame stack (initial `DEFAULT_CALLFRAMES` = 64, hard cap `MAX_CALL_FRAMES` = 1024),
- an exception-context stack of `MAX_FL_EXCEPTION_LEVELS` = 24 nested try/catch/finally regions,
- a one-slot result, a bounded FIFO mailbox, and scheduling state.

Each call frame holds: the executing chunk, the instruction pointer `ip`, up to `MAX_LOCALS` = 128 local slots, the function object, its environment (for closures/globals), and the owning class for methods (`this` dispatch, `super`).

Scheduling is cooperative: the interpreter decrements a per-fiber quantum at scheduling checkpoints (loop back-edges and calls) and yields to the scheduler after `FIBER_QUANTUM` = 10000 checkpoints, or immediately at explicit `OP_YIELD` / `OP_AWAIT` / blocking-I/O suspension points. No OS threads are involved; bytecode never observes preemption in the middle of an instruction.

Implementation note (dispatch): the reference VM runs each opcode as a handler of its own, entered through an `OP_LAST`-sized table, and keeps the instruction pointer, the stack top, the frame's locals and its constants in registers from one instruction to the next. The frame's stored `ip` is only guaranteed current at frame switches, raises and suspension points.

3.2 Calls, frames and returns

Call-site stack layout. Arguments are pushed left-to-right, then the callee on top; `OP_CALL <argc:u8>` consumes all of them and leaves the return value. Method-invocation forms take the receiver from the stack (or a local/global for fused forms) and leave arguments on the stack for the callee.

Arity. A function declares `arity` parameters of which the trailing `optArgs` are optional (`VAL_OPTIONAL` bit). A call MUST satisfy `arity − optArgs ≤ argc ≤ arity`; violations raise `InvalidArgs`. Omitted optionals are filled with `nil` (the real nil value, never an empty slot). Typed parameters (declared type masks) are checked at call time, except by `OP_CALL_SELF` and `OP_TAIL_SELF` (see their entries); `nil` passes any declared type. Arguments bind to `locals[0..argc−1]`; for methods, `locals[0]` is the receiver (`this`) and arguments shift up by one.

Frame entry. Exceeding the frame budget raises a catchable `StackError`; on entry the VM reserves `maxStack` (§8.3) operand slots, plus 20 for the values it pushes itself to call back into a function (arguments, callee, result), so the body cannot overflow mid-execution. A frame that does not fit raises `StackError` before its first instruction. A frame owns its locals and function reference; on return every local and the function reference are released and any try-contexts belonging to the frame are discarded.

Tail calls. `OP_TAIL_CALL` / `OP_TAIL_SELF` replace the current frame instead of pushing (constant call depth); frame-budget overflow is therefore only reachable through non-tail recursion. Tail dispatch to a non-plain callee (builtin, bound method, class, …) degrades to call-then-return. A callee that has native code is entered natively from either form, and the native result becomes the current frame's return value.

Fiber completion. When the last frame returns, the fiber finishes; its return value transfers to the awaiting fiber's result slot if one exists, otherwise it is preserved on the fiber for a later `await`.

Non-callable callees print a console error and produce `nil` — they do **not** raise.

3.3 Fibers, yield and await

Fiber states: `NEW`, `RUNNING`, `SUSPENDED`, `FINISHED`, `WAIT_IO`, `YIELD`, `WAIT_FIBER`. Scheduling checkpoints (quantum decrement) occur at backward jumps (`OP_LOOP`), iterator back-edges and function entries; quantum expiry silently re-queues the fiber (no observable state change). Semantics of `OP_YIELD` / `OP_AWAIT` are in their Chapter 7 entries; the model is: `await fiber` parks the awaiter until the target delivers its result (a finished target delivers immediately); `yield v` delivers `v` to a resumer/awaiter if one is attached, else discards it. An exception that escapes the top of a fiber is handed to the fiber waiting on it - an `await`, or an attached `Fiber.Resume` - and re-raised at that fiber's park point, chaining outward while no handler is found; see §3.4 for a fiber nobody waits on.

3.4 Exceptions

Raises are soft. Raising does not unwind at the raise site: the VM builds an `Exception` instance, stores it as the fiber's current exception, sets the fiber's exception flag, and delivery happens at the next dispatch checkpoint. Consequences an implementer MUST reproduce: an instruction that raises abandons its remaining work, and a raise is only catchable if the fiber has an active try context. A raise that escapes a fiber somebody is waiting on is delivered to that waiter (§3.3); one that escapes a fiber nobody waits on - a detached fiber or the main fiber - prints the stack trace and terminates the **process** (exit code = the exception code), unless the VM is embedded with the no-exit option, in which case the fiber is marked finished and the host continues.

Try contexts. `OP_TRY_BEGIN` pushes a context recording: catch landing pad, finally landing pad, current frame count, current stack depth, the local slot the caught exception binds to, and the owning frame's live local count. Nesting deeper than 24 (`MAX_FL_EXCEPTION_LEVELS`) raises `RuntimeError`. The context carries a state (`in-try` / `in-catch` / `in-finally`) and a pending action (`none` / `raise` / `return` / `break` / `continue`) so that `break` / `continue` / `return` /re-raise all route **through** the finally body.

Delivery. On delivery the VM walks to the innermost context: drains the operand stack to the recorded depth, discards frames above the recorded frame count, resets locals declared after `TRY_BEGIN` to `nil`, then enters the catch (pushing the exception object for `OP_CATCH_BEGIN` to bind) if the context was still in its try body and has a catch — otherwise it enters the finally with a pending re-raise. A raise **inside a finally** discards that context's pending action and propagates outward (the new exception wins). Contexts survive their catch body and pop only at `OP_FINALLY_END`.

Exception codes. The names used by the "Traps" clauses in Chapter 7 map to these numeric codes — observable as the `Code` field of a caught exception, and as the process exit code when uncaught. (Codes marked ° are raised by built-in functions and runtime subsystems rather than by the core instructions; 13 is unassigned.)

Code	Name (Chapter 7)		Code	Name (Chapter 7)
0	`NullPointer`		12	`StackError`
1	`DivisionByZero`		14	° unsafe-mode required
2	`ModuloByZero`		15	`NestingError`
3	`InvalidArgs`		16	° illegal instruction
4	`IndexOutOfBounds`		17	° executable-memory OOM
5	° I/O error		18	° out of fibers
6	`RuntimeError`		19	`ConstAssign`
7	° invalid state		20	° checksum mismatch
8	° out of memory		21	`ClassNonStaticCall`
9	° memory access		22	`TypeMismatch`
10	° array size		23	° assertion failed
11	`GuardNull`		24	`FieldUndeclared`

Const-check caveat. The mutability check used by `OP_SET_GLOBAL`, array fast-path stores, and this-field compound assigns raises `ConstAssign` softly **and the mutation still proceeds**; the exception is delivered at the next checkpoint. `OP_SET_OBJ_L/LL` and `OP_SET_PROPERTY` hard-check instead (no mutation). Independent implementations MUST match this per-site behavior for observable compatibility.

3.5 Modules

A module executes its top-level chunk once; `OP_EXPORT` instructions populate its export table. Export lookup, missing-export errors and the module-object property protocol are specified in the Chapter 7 entries for `OP_EXPORT` and `OP_GET_PROPERTY`. Modules are identified by name + version + fingerprint (§5.3); `library(name, version, hash)` pins all three.

3.6 Self-patching instructions

Five `this.*` instruction forms rewrite themselves in place, at first execution, from hash-keyed lookups into resolved-index fast forms of the **same byte length**:

Emitted form (compiler)	Patched form (runtime)
`OP_GET_THIS_PROP`	`OP_GET_THIS_SLOT` / `OP_GET_THIS_CONST`
`OP_SET_THIS_PROP`	`OP_SET_THIS_SLOT`
`OP_THIS_ARITH_L` / `OP_THIS_ARITH_C`	`OP_THIS_ARITH_SLOT_L` / `OP_THIS_ARITH_SLOT_C`
`OP_THIS_INVOKE`	`OP_THIS_INVOKE_SLOT`

(`OP_INVOKE_INSTANCE` additionally re-writes its inline u16 cache operand; `OP_SUPER_INVOKE` self-patches to `OP_SUPER_INVOKE_SLOT` the same way.) Patching is valid because class member layouts are base-first (§4.5): a resolved index is correct for the defining class and every subclass. Patched operands are resolved indices, **not** constant-pool indices. Patches apply only for non-native classes and happen in the shared in-memory chunk. A static compiler MUST emit only the hash-keyed forms. The loader rejects a `.flx` containing `OP_GET_THIS_SLOT`, `OP_SET_THIS_SLOT`, `OP_GET_THIS_CONST`, `OP_THIS_ARITH_SLOT_L` / `_C`, `OP_THIS_INVOKE_SLOT`, `OP_SUPER_INVOKE_SLOT` or `OP_TAIL_THIS_INVOKE_SLOT`; the three `*_ARR_SLOT_*` forms load, and their field slot is checked on every access. An independent VM MAY treat self-patching as an optional optimization — executing the hash-keyed forms every time is observably equivalent.

3.7 JIT IR side-channel

A chunk may carry a serialized JIT IR section (§5.5) and functions carry JIT-related flags. These are optimization hints only: a conforming VM MAY ignore all of them. The reference VM guards JIT entry behind exact runtime type checks and falls back to interpretation, so JIT presence is never observable in program semantics (timing aside).

The IR is checked for structure at load, not proven safe, so the reference VM lowers it to native code only for a file signed by a trusted key (§5.3); the IR of an unsigned file, or of one signed by an untrusted key, is skipped and its functions run interpreted. A bundled dependency takes the bundle's verdict.

---

4. The value system

4.1 Value types

Every stack slot, local, constant and container element is a *value*. The dynamic type of a value is one of the `ObjectType` bits below. Type words are **bitmasks** (`ValMask`, 32-bit): a single concrete value has exactly one bit set, while declared types (function signatures, typed arrays, `OP_EXPORT` annotations) may union several bits.

Basic storable types (bits 0–8) — permitted as typed-array element types:

Bit	Name	Value	Meaning
0	`VAL_NIL`	`0x00000001`	the single value `nil`
1	`VAL_CHAR`	`0x00000002`	Unicode scalar, 32-bit
2	`VAL_FLOAT`	`0x00000004`	IEEE 754 binary64
3	`VAL_INT`	`0x00000008`	signed 64-bit two's-complement
4	`VAL_STRING`	`0x00000010`	mutable byte string (UTF-8 by convention; `length` is authoritative)
5	`VAL_OBJECT`	`0x00000020`	hash-keyed map (string keys)
6	`VAL_ARRAY`	`0x00000040`	dynamic array
7	`VAL_BOOL`	`0x00000080`	`true` / `false`
8	`VAL_BLOCK`	`0x00000100`	raw memory block / typed buffer (element size 1–255 bytes)
14	`VAL_MATRIX`	`0x00004000`	element modifier, never a value's own type: an array type with this bit and one element type T (`VAL_INT`, `VAL_FLOAT`, `VAL_CHAR` or `VAL_BOOL`) holds only arrays of exactly element type T - the source type `[[T]]`

Bits 9–13 and 15 are reserved for future basic types.

Complex runtime types (bits 16–28) — never element types:

Bit	Name	Value
16	`VAL_FUNCTION`	`0x00010000`
17	`VAL_BUILTIN_FUNCTION`	`0x00020000`
18	`VAL_MODULE`	`0x00040000`
19	`VAL_EXCEPTION`	`0x00080000`
20	`VAL_BOUND_METHOD`	`0x00100000`
21	`VAL_POINTER`	`0x00200000` (internal)
22	`VAL_FIBER`	`0x00400000`
23	`VAL_FFI_FUNCTION`	`0x00800000`
24	`VAL_CLASS`	`0x01000000`
25	`VAL_INSTANCE`	`0x02000000`
26	`VAL_STREAM`	`0x04000000`
27	`VAL_UNKNOWN`	`0x08000000`
28	`VAL_OPTIONAL`	`0x10000000` (parameter may be omitted)

`VAL_ANY` = `0xFFFFFFFF`. Bits 29–31 are reserved.

Typed arrays. A typed array stores its element type in the low 16 bits of its own type word alongside `VAL_ARRAY`, unshifted: `type = VAL_ARRAY | elementBits`. Example: `[int]` has type `0x00000048` (`VAL_ARRAY | VAL_INT`). Arrays typed `[float]` and `[int]` are stored *flat* (`double[]` / `int64_t[]` backing, no per-element boxes); `[float]` writes coerce to float, `[int]` writes accept only ints.

4.2 Numeric model

- **int** is signed 64-bit two's-complement. Arithmetic wraps (implementation note: small ints -2^{60} … $2^{60}-1$ are pointer-tagged immediates, larger ones heap-boxed; the distinction is unobservable).
- **float** is IEEE 754 binary64. Conversions float→int: NaN → 0, $\geq 2^{63}$ → `INT64_MAX`, $< -2^{63}$ → `INT64_MIN`, otherwise truncation toward zero.
- **Division `/` and power `^^` of int by int produce int** (division truncates; power is exact and wraps on overflow); the result is float only if at least one operand is float. `INT64_MIN / -1` and `INT64_MIN % -1` are defined (no trap; result `INT64_MIN` / `0`).
- **Shifts** mask their count to 0–63 (`count & 63`). Left shift is computed in unsigned arithmetic (two's-complement wrap, defined for negative operands); right shift is arithmetic (sign-extending).

- **char** is an unsigned 32-bit Unicode scalar; it participates in arithmetic as an integer and compares numerically.
- **Approximate equality** (`OP_APPROX_EQ`): relative epsilon `1e-2` , absolute epsilon `1e-9` .
- Float formatting uses `%.17g` (round-trip precision).

4.3 nil, truthiness and identity

`nil` is a real value (a singleton), not a null pointer. Reference types may hold `nil` . Truthiness rules are used by `OP_JUMP_IF_FALSE` / `OP_JUMP_IF_TRUE` , `OP_NOT` and the logical operators (see §7, OP_JUMP_IF_FALSE, for the normative table). `OP_CMP_IS` compares identity (same object), not structural equality.

4.4 Strings

Strings are **mutable in place**, length-tracked byte sequences, stored with a trailing NUL. `length` is authoritative: a constant never contains a NUL byte, but a string built at runtime from binary input (a buffer, a file, a host's explicit-length string) may, and every string operation is bounded by `length` , not by the first NUL. Implementation notes: strings of ≤ 7 bytes use small-string optimization; a string caches its 64-bit hash and must rehash after in-place writes.

4.5 Objects, classes and instances

- A plain **object** (`VAL_OBJECT`) is a hash map with string keys.
- A **class** carries an ordered, kind-tagged member table (`FIELD` , `METHOD` , `CONST`) declared base-first; inheritance nests at most `MAX_PROTO_NEST` = 8 deep. A resolved layout is cached per class: an inherited member keeps its index and field slot in every subclass.
- An **instance** is slot-backed: a flat array of field values, one slot per `FIELD` member, addressed by the field-slot index resolved through the class layout. Instances are **sealed** — only declared fields exist.
- Property names are addressed by **64-bit hash** at runtime (§4.6); there is no interned-symbol table.

4.6 Name hashing

Identifiers (globals, property keys, method names) are referenced in bytecode operands as indices into the chunk's constant pool; the runtime resolves them by **xxHash64(name, seed = 0)** over the identifier bytes. The empty string hashes to `0xEF46DB3751D8E999` . There is no separate table of well-known names to implement: every hash is derivable from the identifier text (e.g. `Constructor` hashes to `0x9A9C0E93AC390B36` — a useful self-test vector). An independent implementation MUST reproduce xxHash64 exactly to interoperate with compiled chunks and native modules.

4.7 Reference counting and cycles

Memory is reclaimed by **pure reference counting** with deferred frees. There is no tracing collector and no cycle collector: a reference cycle is never reclaimed. This is an observable property of the machine (destructor timing, `--mem` reporting) and a documented language-level constraint, not a defect an implementation must reproduce bug-for-bug; an implementation MAY reclaim cycles but SHOULD document the deviation.

---

5. `.flx` container format

5.1 File layout

```
offset 0      FLS header, 192 bytes                    (§5.2)
offset 192    [bundles only] TOC: u16 count + entries  (§5.7)
              main section:  string pool  (§5.4)
                             top-level chunk (§5.5)
              [bundles only] dependency sections
```

A non-bundle file's main section MUST start exactly at offset 192. All region lengths are derived (`end − start`); none is stored explicitly.

5.2 The FLS header (192 bytes)

Magic is the four ASCII bytes `FLS2`. Total header size: 192 bytes (`FLS_HEADER_SIZE`).

Offset	Size	Field	Type	Meaning
0	4	`magic`	raw	`"FLS2"` = `46 4C 53 32`
4	4	`version`	u32	**library version**, packed `0xMMmmppbb` (major.minor.patch.build). Default `0x01000000` (1.0.0.0). Informational: not validated at load
8	4	`compilerVersion`	u32	**bytecode version** = `SYS_VERSION`. Loader MUST enforce `MIN_SUPPORTED_BYTECODE_VERSION ≤ compilerVersion ≤ SYS_VERSION`
12	32	`name`	UTF-8	NUL-terminated output basename *without extension*, ≤ 31 chars. Part of the signed/fingerprinted bytes: renaming the output changes the fingerprint
44	4	`flags`	u32	bit 0 = `FLAG_BUNDLE`; all other bits reserved: writers set them to 0 and a reader MUST reject a file with any other bit set
48	8	`entry`	u64	xxHash64 of the entry function name — `0xC6783A229050BEDC` (`"Main"`) if present, `0` for a library with no entry point
56	4	`mainSectionStart`	u32	absolute offset; MUST be 192 for non-bundles, `192 + 2 + depCount·152` for bundles
60	4	`stringPoolStart`	u32	= `mainSectionStart` in the current writer
64	4	`stringPoolEnd`	u32	
68	4	`codeStart`	u32	= `stringPoolEnd` in the current writer
72	4	`codeEnd`	u32	= `mainSectionEnd` in the current writer
76	4	`mainSectionEnd`	u32	
80	2	—		reserved; MUST be zero, a reader MUST reject nonzero
82	1	`sigAlg`	u8	0 = unsigned, 1 and 2 = retired schemes (MUST be rejected), 3 = Ed25519 (§5.3). Any other value MUST be rejected
83	1	—		reserved; MUST be zero, a reader MUST reject nonzero
84	32	`pubkey`	raw	Ed25519 public key; all-zero when unsigned
116	64	`signature`	raw	Ed25519 signature; all-zero when unsigned
180	12	—		reserved; MUST be zero, a reader MUST reject nonzero

The four section-offset pairs are validated for ordering (`mainSectionStart ≤ stringPoolStart ≤ stringPoolEnd ≤ codeStart ≤ codeEnd ≤ mainSectionEnd ≤ fileSize`) but the parser reads pool and chunk **contiguously** from `mainSectionStart`; `codeStart` / `codeEnd` exist for validation and future extensibility only.

5.3 Signature and trust

Algorithm. Ed25519 as in RFC 8032, with SHA-512. The 64-byte secret key is `seed ‖ pubkey`; a signer derives the public half from the seed, so the header always carries the key this scheme gives that seed. Scheme 2 used the same curve arithmetic with BLAKE2b in place of SHA-512 and is retired.

Signed content. `digest = SHA-256( file[0..116) ‖ 64 zero bytes ‖ file[180..EOF) )` — the entire file with the 64-byte signature field masked to zero. `sigAlg` and `pubkey` are inside the signed range, so neither can be altered without breaking the signature.

Domain separation. The message actually signed is the 50-byte concatenation `"flaris.flx.sig.v1" ‖ 0x00 ‖ digest` (context string including its terminating NUL, 18 bytes, followed by the 32-byte digest).

Signing flow. The file is written with the signature field zeroed, hashed in one pass (equal to the masked digest by construction), signed, and the 64 signature bytes patched in place at offset 116.

Fingerprint. The module fingerprint (printed at compile time, pinned by `library(name, version, hash)` and by the package manager) is the SHA-256 of the **complete final file including the signature** — identical to `sha256sum file.flx`.

Verification verdicts and load policy. A loader MUST classify a file as one of: `UNSIGNED` (sigAlg 0, key/sig fields zero), `VALID` (cryptographically valid, signer not trusted), `TRUSTED` (valid and signer key in the trust set), `BAD` (signature check failed), `STRIPPED` (sigAlg 0 but key/signature bytes nonzero — tamper evidence), `UNSUPPORTED` (sigAlg 1, 2 or any unknown value). `BAD`, `STRIPPED` and `UNSUPPORTED` MUST always be rejected. `UNSIGNED` and `VALID` are rejected only under `--require-signed`. All rejects exit with code 4. The reference trust set is one built-in anchor (`flaris-lang.org`, `d43ec4260fe82ee101a939caabff95358b0698718d7b8c38652b5457bdf59621`) plus `~/.flaris/trusted_keys` (path override `$FLARIS_TRUSTED_KEYS`; one `hex64 [label]` per line, `#` comments, ≤ 256 keys, unreadable file = fail closed).

5.4 String pool

Each code region (the main section, and each bundled dependency section) begins with a shared string pool that deduplicates strings referenced more than once in the region's value tree:

```
u16   count               (0 … 65535)
count × {
    u8    isStatic        (informational; reader ignores it)
    u64   hash            (xxHash64 of body; the reader recomputes it and MUST reject a mismatch)
    u32   len             (MUST be < MAX_STRING_SIZE)
    u8[len] body          (no NUL terminator)
}
```

Wherever a serialized value would be a string that has a pool slot, it is replaced by the 6-byte reference `u32 0xFFFFFF01` (`FLX_POOL_REF`) + `u16 slot`. `0xFFFFFF01` does not collide with any `ObjectType` tag, so the reader's type-tag switch stays unambiguous. `slot ≥ count` MUST be rejected.

Writer policy (informative): a string is pooled only if it occurs more than once (keyed on exact bytes + length + static flag — never on the cached hash, which may be stale for mutable strings); slots are assigned in first-

seen order; only leaf strings are pooled (functions can't be — their nested chunk would need the pool before it is complete); hash-only keys (§5.6) are never pooled.

5.5 Chunk serialization

A *chunk* is one function body (the top level is a chunk too). On the wire:

Chunk header — 13 bytes:

Offset	Size	Field	Validation
0	1	magic	`0xA5` (`CHUNK_MAGIC`)
1	4	version u32	MUST satisfy `MIN_SUPPORTED_BYTECODE_VERSION ≤ v ≤ CHUNK_VERSION`
5	2	consts u16	MUST be ≤ `MAX_CONST_COUNT` (16384)
7	4	codelen u32	MUST be ≥ 1 and ≤ `MAX_CODE_SIZE` (10 MiB)
11	2	flags u16	bit 0 `CHUNK_FLAG_DBG` , bit 1 `CHUNK_FLAG_HAS_JIT_IR` , bit 2 `CHUNK_FLAG_HAS_IMPORTS` ; any other bit MUST be rejected

Chunk body, immediately following:

1. `consts` serialized constant values, in pool-index order (§5.6);
2. `codelen` bytes of code (§6);
3. if `CHUNK_FLAG_DBG` : `u8 localCount` (≤ 128), then `localCount × { u32 len; u8[len] name }` — local-variable names;
4. if `CHUNK_FLAG_HAS_JIT_IR` : `u8 regCount` , `u32 irLen` (≤ 10 MiB), `u8[irLen]` JIT IR. This section is an optional side-channel: a conforming VM MAY skip it entirely. The stored `regCount` MUST NOT be trusted (the reference VM recomputes it from the IR); semantically invalid IR is dropped (the function runs interpreted) and is *not* a load error.
5. if `CHUNK_FLAG_HAS_IMPORTS` : `u16 count` , then `count × { u16 len; u8[len] name; u16 len; u8[len] version; u16 len; u8[len] pin }` — one record per top-level `import ... from library(name, version[, pin])` declaration, in source order, `pin` empty when unpinned. Lengths MUST be below 512 / 64 / 80 respectively. Only a top-level chunk carries the section; it is what a bundler resolves dependencies from.

Debug info is off by default. Compiling with `-g` (alias `--no-strip`) sets `CHUNK_FLAG_DBG` , writes the debug section and emits `OP_DBG_LINE` ; `--strip` restates the default (it also overrides the `-g` that `--debug` implies). `OP_DBG_FILE_NAME` and `OP_DBG_BREAK` are emitted either way. There is no separate line table: source lines are the inline `OP_DBG_LINE` instructions.

5.6 Constant value serialization

General wire form: `u32 typeTag` + `u8 isStatic` + payload. The type tag is the value's `ObjectType` bit (§4.1). Exception: a pool reference (§5.4) is `u32 0xFFFFFF01` + `u16 slot` with **no** isStatic byte. The reader ignores `isStatic` in all cases.

Tag	Payload
`VAL_NIL 0x01`	none
`VAL_CHAR 0x02`	u32 codepoint
`VAL_FLOAT 0x04`	f64 (IEEE 754 bits, LE)
`VAL_INT 0x08`	i64

Tag	Payload
`VAL_STRING 0x10`	`u64 hash` + `u32 len` + `u8[len]` body. `len ≥ MAX_STRING_SIZE` MUST be rejected. If `len > 0` the reader recomputes the hash (stored hash ignored); if `len == 0` the stored hash is preserved — this is the **hash-only key** form
`VAL_OBJECT 0x20`	`u32 count` (< `MAX_ITEMS`), then `count ×` { key value — MUST decode to a string; value }. The writer emits pairs in insertion order, so a constant object keeps its source key order and identical source gives identical bytes
`VAL_ARRAY 0x40`	`u32 length` (< `MAX_ITEMS`) + `u8 flat`. `flat = 1`: `length × f64` (flat float array, §4.1); `flat = 2`: `length × i64` (flat int array); `flat = 0`: `length` nested values. Any other `flat` value MUST be rejected
`VAL_BOOL 0x80`	u8 (0/1)
`VAL_BLOCK 0x100`	none — blocks do not serialize; reads back as `nil`
`VAL_FUNCTION 0x10000`	`u8 arity` + `u8 localCount` + `u32 flags` + `u32 returnType` + `16 × u32 argsTypes` + name (value-key, may be the nil sentinel) + **nested chunk** (§5.5, recursive). Reader MUST clamp `localCount ≤ 128`, `arity ≤ 16`, clear flags `FUNC_FLAG_IS_CLONE` + `FUNC_FLAG_ENV_OWNED`, and re-derive `FUNC_FLAG_TYPED_PARAMS` itself
`VAL_BUILTIN_FUNCTION` / `VAL_MODULE` / `VAL_EXCEPTION` / `VAL_BOUND_METHOD` / `VAL_POINTER`	none — read back as `nil`
`VAL_CLASS 0x1000000`	`u32 ownCount` (≤ 65535) + `u64 protoHash` (base-class name hash, 0 = none) + name (must be string) + `ownCount ×` { `u8 kind` (0 field / 1 method / 2 const, in the low 7 bits; any other value MUST be rejected), member name (must decode to string), member value, and when bit 7 (`0x80`) of the kind is set: `u32 fieldType` }. Bit 7 is valid on a field only; `fieldType` is the `VAL_*` mask every store into the field must fit (§4.5), and a field without it is untyped. Function members are back-linked to the class as owner
any other tag	MUST be rejected

Value-keys (object keys, class member names, function names) are ordinary serialized values, with two special forms: a NULL name is the 5-byte sentinel `[u32 VAL_NIL][u8 0]`, and under size optimization (`--small`) a string key may be written **hash-only**: `[u32 VAL_STRING][u8 static][u64 hash][u32 len=0]` — identity survives (runtime identity is the hash, §4.6), the text does not.

Nesting depth. The loader tracks one depth counter shared by chunk and value deserialization, rejecting at 64 (`MAX_CHUNK_DEPTH`). A nested function costs **two** levels (value → its chunk), so function nesting deeper than ~31 levels is not loadable.

5.7 Bundles

A bundle packs a program with its library dependencies into one file. Header bit `FLAG_BUNDLE` is set and a table of contents follows the header:

```
@192   u16 entryCount            (MUST be ≤ 1024)
@194   entryCount × BundleEntry  (152 bytes each)
       main section              (at hdr.mainSectionStart = 194 + entryCount·152)
       dependency sections
```

BundleEntry — 152 bytes:

Offset	Size	Field	Meaning
0	64	`name`	module name, NUL-terminated, `.flx` stripped
64	4	`version` u32	dependency's library version (`0xMMmmppbb`)
68	8	`offset` u64	absolute file offset of the dependency's section
76	4	`size` u32	section length (pool + chunk)
80	32	`sha256`	whole-file SHA-256 of the original dependency `.flx` — the pin value for `library(name, ver, hash)`
112	1	`sigAlg` u8	dependency's original signature algorithm
113	32	`pubkey`	dependency's signing key (zero if unsigned)
145	7	—	reserved, zero

A dependency section is a byte-copy of `[mainSectionStart, mainSectionEnd)` of the original dependency file — its pool and chunk travel together, so pool references stay valid. Per-dependency bounds errors (offset/size outside the file, size 0) skip that dependency with an error rather than aborting the load. Under `--require-signed` the TOC-recorded `sigAlg` / `pubkey` hold bundled dependencies to the same trust policy as standalone files (the dependency's own header does not survive bundling); the TOC itself is covered by the bundle's outer signature. A bundle builder MUST refuse dependencies whose signatures verify as `BAD`, `STRIPPED` or `UNSUPPORTED`.

Load order. A bundled dependency may import another dependency in the same bundle. A loader MUST therefore make every TOC entry resolvable before executing any of them — registering each as an unloaded module and running it on first import — so that a dependency can import one listed after it. Trust (`--require-signed`) is decided for every entry at load time, whether or not anything imports it; content pins are checked before the dependency executes, using the `sha256` the TOC already carries. A dependency nothing imports is never executed. A cycle between bundled dependencies is detected and reported rather than recursing without bound; the import that closes the cycle fails, so the dependency holding it is left partially initialised.

Independently, a bundle builder SHOULD write TOC entries in dependency order — every dependency ahead of the entries that import it — so the bundle also loads on a runtime that walks the TOC front-to-back. Entries carry explicit offsets, so TOC order is independent of where the sections sit in the file. Order is advisory: it cannot be relied on, since nothing prevents a hand-built TOC from listing entries arbitrarily.

---

6. Instruction encoding

6.1 General form

An instruction is a 1-byte opcode followed by zero or more fixed-width operands in the order given by its definition (§7). Multi-byte operands are little-endian. There is no alignment: instructions pack back-to-back. Exactly two instructions have variable length (`OP_FN_CAPTURE` and `OP_JUMP_TABLE` , §6.5); every other instruction's length is a static function of its opcode.

6.2 Operand kinds

Kind	Width	Meaning
`const:u16`	2	index into the chunk constant pool (< `constCount` ≤ `MAX_CONST_COUNT` = 16384)
`key:u16`	2	constant-pool index of an identifier string (resolved to its xxHash64 at run time)
`slot:u8`	1	local-variable slot (< frame `localCount` ≤ `MAX_LOCALS` = 128)
`argc:u8`	1	argument count (≤ `MAX_CALL_ARGUMENTS` = 16)

Kind	Width	Meaning
`aop:u8`	1	arithmetic sub-operator (§6.3)
`cop:u8`	1	comparison sub-operator (§6.4)
`imm:i8` / `imm:i16` / `imm:i32`	1/2/4	signed integer immediate
`imm:f32`	4	IEEE binary32 immediate, widened to binary64 on push
`imm:u32`	4	unsigned immediate (`OP_IMM_CHAR` codepoint)
`off:u16`	2	branch displacement (forward for `JUMP*`, backward for `LOOP`), relative to the address after the operand (§6.6)
`addr:u16`	2	absolute code offset within the chunk
`type:u16` / `type:u32`	2/4	`ValMask` bits (§4.1): array element type, export type

6.3 Arithmetic sub-operators (`ArithOp`)

Used by every `OP_ARITH_*` form except `OP_ARITH_FMA_LLL` , and by the `OP_THIS_ARITH_*` and `OP_OBJ_ARITH_*` forms:

Value	Name	Operator
0	`ARITH_ADD`	`+`
1	`ARITH_SUB`	`-`
2	`ARITH_MUL`	`*`
3	`ARITH_DIV`	`/`
4	`ARITH_MOD`	`%`
5	`ARITH_POW`	`^^`
6	`ARITH_AND`	`&`
7	`ARITH_OR`	`|`
8	`ARITH_XOR`	`^`
9	`ARITH_SHL`	`<<`
10	`ARITH_SHR`	`>>`
11	`ARITH_USHR`	`>>>`

A loader MUST reject `aop` ≥ 12 (`ARITH_LAST`).

6.4 Comparison sub-operators

Used by `OP_CMP_JUMP_LL/LC/LC32` . **The branch fires when the comparison is false** (these ops fuse `compare + JUMP_IF_FALSE`):

Value	Name	Condition
0	`CMP_EQ`	`==`
1	`CMP_NEQ`	`!=`
2	`CMP_LT`	`<`
3	`CMP_GT`	`>`
4	`CMP_LE`	`<=`
5	`CMP_GE`	`>=`

6.5 Instruction lengths

Total length in bytes, opcode included. This table is normative for stepping over code; a decoder MUST treat a truncated instruction (operands running past `codelen`) as malformed.

Every opcode appears in exactly one row, in opcode-number order within the row.

Len	Instructions
1	`NOP`, `NIL`, `TRUE`, `FALSE`, `POP`, `DUP`, `NEGATE`, `NOT`, `BITWISE_NOT`, `ADD`, `SUBTRACT`, `MULTIPLY`, `DIVIDE`, `MODULO`, `POWER`, `BITWISE_AND`, `BITWISE_OR`, `XOR`, `SHL`, `SHR`, `USHR`, `IS_NIL`, `IS_NOT_NIL`, `EQUAL`, `NOT_EQUAL`, `LESS`, `LESS_EQUAL`, `GREATER`, `GREATER_EQUAL`, `APPROX_EQ`, `CMP_IS`, `GET_LOCAL0`, `GET_LOCAL1`, `GET_LOCAL2`, `GET_LOCAL3`, `SET_LOCAL0`, `SET_LOCAL1`, `SET_LOCAL2`, `SET_LOCAL3`, `GET_INDEX`, `SET_INDEX`, `MAKE_CONST`, `RETURN`, `RETURN_NONE`, `RETURN_NIL`, `YIELD`, `AWAIT`, `BIND_THIS`, `TRY_END`, `CATCH_BEGIN`, `CATCH_END`, `FINALLY_BEGIN`, `FINALLY_END`, `THROW`, `LEN`, `TYPE`, `HAS_KEY`, `TO_STRING`, `TO_INT`, `TO_FLOAT`, `TO_CHAR`, `TO_I8`, `TO_U8`, `TO_I16`, `TO_U16`, `TO_I32`, `TO_U32`, `SUPER`, `GUARD`, `CONCAT`
2	`IMM8`, `GET_LOCAL`, `SET_LOCAL`, `INC_LOCAL`, `DEC_LOCAL`, `GET_ARR_LI`, `GET_INDEX_LOCAL`, `SET_INDEX_LOCAL`, `CALL`, `CALL_SELF`, `TAIL_SELF`, `TAIL_CALL`, `RETURN_L`, `NEW`, `GET_BLK_LI`, `CONCAT_N`, `NIL_LOCAL`, `CALL_WITH_THIS`
3	`CONSTANT`, `IMM16`, `JUMP_IF_FALSE`, `JUMP_IF_TRUE`, `JUMP`, `LOOP`, `DEFINE_GLOBAL`, `GET_GLOBAL`, `SET_GLOBAL`, `ARITH_L`, `ARITH_IMM8`, `GET_PROPERTY`, `SET_PROPERTY`, `GET_ARR_LC`, `GET_ARR_LL`, `SET_ARR_LC`, `SET_ARR_LL`, `GET_THIS_PROP`, `SET_THIS_PROP`, `FN`, `PUSH_BUILTIN`, `CALL0_BUILTIN`, `CALL1_BUILTIN`, `CALL2_BUILTIN`, `CALL3_BUILTIN`, `CALL4_BUILTIN`, `CALL5_BUILTIN`, `DBG_LINE`, `DBG_FILE_NAME`, `DBG_BREAK`, `GET_BLK_LL`, `SET_BLK_LL`, `GET_THIS_SLOT`, `SET_THIS_SLOT`, `GET_THIS_CONST`, `CONSTANT_REF`
4	`ARITH_LC`, `ARITH_LL`, `ARITH_L_IMM8`, `ARITH_LL_PUSH`, `SET_ARR_LLC`, `SET_ARR_LLL`, `SET_OBJ_L`, `LOCAL_ARR_LC`, `LOCAL_ARR_LL`, `THIS_INVOKE`, `SET_BLK_LLL`, `GET_LOCAL_PROP`, `THIS_INVOKE_SLOT`, `SUPER_INVOKE`, `TAIL_THIS_INVOKE`, `TAIL_THIS_INVOKE_SLOT`, `SUPER_INVOKE_SLOT`, `ARITH_IMM16` (aop:u8 imm:i16), `CMP_C`, `GET_MAT_LLL`, `SET_MAT_LLL`
5	`IMM32`, `IMM_CHAR` (u32 codepoint), `ARITH_ELC`, `ARITH_ELL`, `ARITH_EIC`, `ARITH_EIL`, `SET_OBJ_LL`, `THIS_ARITH_L`, `THIS_ARITH_C`, `BUILD_OBJECT`, `BUILD_ARRAY` (count:u16 + elemType:u16), `TRY_LEAVE`, `ITER_BEGIN`, `ITER_NEXT`, `ARITH_FMA_LLL`, `THIS_ARITH_SLOT_L`, `THIS_ARITH_SLOT_C`, `CALL_GLOBAL`, `OBJ_ARITH_S`
6	`CMP_JUMP_LL`, `CMP_JUMP_LC`, `TRY_BEGIN`, `INVOKE_INSTANCE` (key:u16 argc:u8 slotCache:u16 — the cache field is runtime-managed scratch), `OBJ_ARITH_L` (obj:u8 key:u16 aop:u8 src:u8), `ARITH_MLL` (m:u8 i:u8 j:u8 aop:u8 src:u8), `ARITH_MLC` (m:u8 i:u8 j:u8 aop:u8 imm:i8)
7	`FOREACH_BEGIN` (slot1:u8 slot2:u8 iterable:u8 index:u8 exitAddr:u16; slot operands may be the sentinel `0xFF` = unused), `FOREACH_NEXT` (the same slots, back:u16), `CHECK_TYPE` (mask:u32 label:u16)
9	`EXPORT` (aliasIdx:u16 nameIdx:u16 type:u32), `CMP_JUMP_LC32`, `PUSH_HOST`, `CALL0_HOST`, `CALL1_HOST`, `CALL2_HOST`, `CALL3_HOST`, `CALL4_HOST`, `CALL5_HOST` (host forms: modHash:u32 fnHash:u32, rewritten in place to one member pointer at load)
11	`INVOKE_MEMBER` (key:u16 argc:u8 mode:u8 classIdGuard:u32 idx:u16 — the last three fields are runtime-managed cache, emitted as zero)
13	`INVOKE_GLOBAL_CACHED` (globalKey:u16 methodKey:u16 argc:u8 mode:u8 classIdGuard:u32 idx:u16 — cache fields as in INVOKE_MEMBER)
var	`JUMP_TABLE` = 12 + 2·size: `min:i32 max:i32 size:u8`, then size × `caseOff:u16`, then `defaultOff:u16` (each a distance forward from the first table entry). `FN_CAPTURE` = 4 + 3·count: `fnIdx:u16 count:u8`, then count × (`nameConstIdx:u16 parentSlot:u8`)

6.6 Branch-target computation

Let `base` be the code offset immediately **after** the instruction's last operand byte.

- `OP_JUMP`, `OP_JUMP_IF_FALSE`, `OP_JUMP_IF_TRUE`, `OP_CMP_JUMP_LL/LC/LC32`: `target = base + off` (forward).
- `OP_LOOP`, `OP_ITER_NEXT`, `OP_FOREACH_NEXT`: `target = base − off` (backward).
- `OP_JUMP_TABLE`: every table entry is `target − tableStart`, where `tableStart` is the offset of the first entry (forward).

- `OP_ITER_BEGIN`, `OP_FOREACH_BEGIN`, `OP_TRY_BEGIN` (`catchIp`/`finallyIp`), `OP_TRY_LEAVE`: operands are **absolute** code offsets within the chunk.

---

7. Instruction set reference

Entries are in enum order (opcode numbers in Appendix A). Format:

> `MNEMONIC <operand:type> …` — total length Stack `…, a, b → …, r` (right end = TOS) · Operation · Traps

"Traps: —" means the instruction cannot raise. All raises are soft (§3.4). Fused instructions are semantically equivalent to the generic sequence they replace unless a difference is stated.

7.1 Stack and constants

`OP_NOP` — 1 byte. Stack `→`. No operation. Traps: —

`OP_CONSTANT` `<idx:const:u16>` — 3 bytes. Stack `→ v`. Pushes constant `idx`. A **string** constant is pushed as a fresh copy (strings are mutable; the pool object must not be aliased); other values are pushed by reference. Traps: —

`OP_CONSTANT_REF` `<idx:const:u16>` — 3 bytes. Stack `→ v`. Pushes constant `idx` by reference, a string included. The compiler emits it for a string literal whose consumer only reads it: an argument of a builtin call (`OP_CALLn_BUILTIN`) whose builtin is pure and returns a scalar or an always-new string or block; an operand of a comparison, of `in` or `is`, of a `+` proven to be text concatenation or of an `OP_CONCAT_N` chain; the key of an index read; a `switch` case label. Nothing can keep or change the literal there; string constants are immutable, so a write raises instead of reaching the pool. Traps: —

`OP_NIL` / **`OP_TRUE`** / **`OP_FALSE`** — 1 byte. Stack `→ v`. Push `nil` / `true` / `false`. Traps: —

`OP_IMM8` `<v:i8>` / **`OP_IMM16`** `<v:i16>` / **`OP_IMM32`** `<v:i32>` — 2/3/5 bytes. Push the sign-extended int. Traps: —

`OP_IMM_CHAR` `<cp:u32>` — 5 bytes. Push char with codepoint `cp`. Traps: —

`OP_POP` — 1 byte. Stack `v →`. Discards TOS. Traps: `StackError` (underflow — unreachable in validated code).

`OP_DUP` — 1 byte. Stack `v → v, v`. Traps: `StackError`.

7.2 Arithmetic and logic

`OP_NEGATE` — 1 byte. Stack `v → −v`. Int negation wraps (`−INT64_MIN = INT64_MIN`); float negates. Traps: `InvalidArgs` on non-numeric.

`OP_NOT` — 1 byte. Stack `v → b`. Pushes `!truthy(v)` (§7.4 truthiness). Traps: —

`OP_BITWISE_NOT` — 1 byte. Stack `v → ~v`. Int only. Traps: `InvalidArgs`.

`OP_ADD` — 1 byte. Stack `a, b → r`. Lane selection, in order:

1. int ⊕ int (bool/char count as int): 64-bit wrapping add;
2. either float: binary64 add;
3. char + char: a **string** of the two codepoints (UTF-8);
4. object + object: clone of `a` with `b`'s properties merged over it;
5. array + array: clone of `a` with `b`'s elements appended; array + scalar: clone of `a` with the scalar appended;
6. anything else: **string concatenation** — both operands coerced to text. Traps: `NestingError` (deep container merge).

`OP_SUBTRACT` / **`OP_MULTIPLY`** — 1 byte. Stack `a, b → r`. Int lane wraps; float lane if either operand is float. Traps: `InvalidArgs` on non-numeric operands.

`OP_DIVIDE` — 1 byte. Stack `a, b → r`. int/int: truncating division, result int; `INT64_MIN / −1 = INT64_MIN`. Float lane if either is float. Zero divisor raises `DivisionByZero`. Note: unless both operands are small ints, the zero check reads the divisor as a number first, and a non-numeric value reads as 0.0 — so dividing by a non-numeric value raises `DivisionByZero`, not a type error.

`OP_MODULO` — 1 byte. Stack `a, b → r`. int lane: C remainder (sign of the dividend: `−7 % 3 = −1`); `INT64_MIN % −1 = 0`. Float lane: `fmod`. Zero divisor raises `ModuloByZero` (same coercion note as `OP_DIVIDE`).

`OP_POWER` — 1 byte. Stack `a, b → r`. int ^^ int: exact 64-bit integer power, wrapping on overflow like `OP_MULTIPLY`; a negative exponent gives 0 except for the bases 1 and -1. Float lane if either is float. Traps: `InvalidArgs`.

`OP_BITWISE_AND` / **`OP_BITWISE_OR`** / **`OP_XOR`** — 1 byte. Stack `a, b → r`. Int-only bitwise ops. Traps: `InvalidArgs` ("requires integer operands").

`OP_SHL` / **`OP_SHR`** / **`OP_USHR`** — 1 byte. Stack `a, b → r`. Shift count masked `& 63`; `<<` computed in unsigned (defined two's-complement wrap); `>>` is arithmetic (sign-extending); `>>>` is logical (zero-filling) on the same 64 bits. Int-only. Traps: `InvalidArgs`.

7.3 Comparison

`OP_IS_NIL` / **`OP_IS_NOT_NIL`** — 1 byte. Stack `v → b`. Tests `v` against the nil singleton. Traps: —

`OP_EQUAL` / **`OP_NOT_EQUAL`** / **`OP_LESS`** / **`OP_LESS_EQUAL`** / **`OP_GREATER`** / **`OP_GREATER_EQUAL`** — 1 byte. Stack `a, b → r:bool`. Rules:

- identity short-circuit: `a == a` is equal;
- numeric lane: int/float/char/bool inter-comparable (double compare if either is float, else int64);
- strings: equality is length + byte compare (the cached hash is only a fast-reject hint, never trusted alone); ordering is byte-lexicographic, shorter-is-less on a common prefix;
- arrays: element-wise structural equality; ordering by first differing element, else shorter-is-less;
- blocks: raw byte compare (equality);
- `nil == nil` is true;
- **incompatible types**: `!=` is true; every ordering comparison is false. Traps: `NestingError` beyond depth 1024 (`MAX_COMPARE_DEPTH`).

`OP_APPROX_EQ` (`~=`) — 1 byte. Stack `a, b → r:bool`. string ~= string: case-insensitive equality. Numeric: NaN never equal, infinities equal only same-signed, else `|a−b| ≤ 1e−9 + 1e−2·max(|a|,|b|)`. Other

operands → false. Traps: —

`OP_CMP_IS` — 1 byte. Stack `v, t → r:bool`. If `t` is a class: instance-of / subclass-of test of `v` (proto chain, ≤ 8 deep), so a class `v` tests as a subclass and `Base is Derived` is false. Otherwise plain identity `v == t`. Traps: —

`OP_CMP_C` `<const:u16> <cop:u8>` — 4 bytes. Stack `a → r:bool`. Compares `a` with `constants[const]` per `cop` (§6.4), by the rules of `OP_EQUAL` and its siblings above. The constant is read where it sits and never pushed, so a string literal is not copied. Not compiler-emitted from source: the peephole pass writes it over `OP_CONSTANT` followed by a comparison, which has the same length. A `cop` above 5 is rejected at load. Traps: `NestingError` (deep structural compare).

7.4 Control flow

`OP_JUMP_IF_FALSE` / **`OP_JUMP_IF_TRUE`** `<off:u16>` — 3 bytes. Stack `c →`. Branch forward (§6.6) when `c` is falsy / truthy. **Truthiness** (normative): `nil` → false; bool → itself; int/float/char → `≠ 0`; string → non-empty; array → `length > 0`; object → has ≥ 1 property; every other type (functions, classes, instances, fibers, …) → true. Traps: —

`OP_JUMP` `<off:u16>` — 3 bytes. Unconditional forward branch. Traps: —

`OP_LOOP` `<off:u16>` — 3 bytes. Unconditional **backward** branch; the mandatory loop back-edge checkpoint (quantum, §3.3). Traps: —

`OP_JUMP_TABLE` `<min:i32> <max:i32> <size:u8> size×<case:off:u16> <default:off:u16>` — 12 + 2·size bytes. Stack `v →`. Coerces `v` to int; if `min ≤ v ≤ max` and `v − min < size` jumps to `case[v − min]`, else to `default`. Every entry is the target's distance forward from the first table entry (§6.6). A label repeated in the switch occupies its table slot once, holding the FIRST body that declared it - the same case the comparison-chain lowering would run. Traps: —

`OP_CMP_JUMP_LL` `<slotA:u8> <slotB:u8> <cop:u8> <off:u16>` — 6 bytes. Stack `→`. Compares `local[slotA]` with `local[slotB]` per `cop` (§6.4) and branches forward **when the condition is false**. Zero stack traffic. Equivalent to `GET_LOCAL a; GET_LOCAL b; CMP; JUMP_IF_FALSE`. Traps: `NestingError` (deep structural compare).

`OP_CMP_JUMP_LC` `<slot:u8> <cop:u8> <imm:i8> <off:u16>` — 6 bytes. As `OP_CMP_JUMP_LL` with an i8 immediate right operand. A small-int local compares as int64, a float local as a double against `(double)imm`; any other value (nil, bool, char, string, container, …) compares exactly as the unfused `GET_LOCAL; IMM8; CMP` sequence would, so `nil == 0` and `"abc" < 5` are false. Traps: —

`OP_CMP_JUMP_LC32` `<slot:u8> <cop:u8> <imm:i32> <off:u16>` — 9 bytes. `OP_CMP_JUMP_LC` with an i32 immediate. Traps: —

7.5 Globals and locals

`OP_DEFINE_GLOBAL` `<key:u16>` — 3 bytes. Stack `v →`. Defines the name in the current environment. Scalar values are stored as private copies (value semantics); containers by reference. Redefinition/shadowing prints a warning but succeeds. Traps: —

`OP_GET_GLOBAL` `<key:u16>` — 3 bytes. Stack `→ v`. Resolves the name hash through the environment chain (closure env → module env → global env). **An undefined global yields `nil` — no raise.** Traps: —

OP_SET_GLOBAL `<key:u16>` — 3 bytes. Stack `v →`. Rebinds the nearest existing binding (may rebind through parent environments). Scalars copied. Assigning over a `const` binding raises `ConstAssign` softly — **the store still happens** and the exception arrives at the next checkpoint (§3.4).

OP_GET_LOCAL `<slot:u8>` / **OP_GET_LOCAL0 … OP_GET_LOCAL3** — 2 / 1 bytes. Stack `→ v`. Pushes `local[slot]` (slots 0–3 have dedicated 1-byte forms). Traps: —

OP_SET_LOCAL `<slot:u8>` / **OP_SET_LOCAL0 … OP_SET_LOCAL3** — 2 / 1 bytes. Stack `v →`. Stores into the slot, releasing the previous occupant; scalars copied (value semantics). Writing a slot ≥ the frame's live-local count extends the count. Traps: —

OP_INC_LOCAL / OP_DEC_LOCAL `<slot:u8>` — 2 bytes. Stack `→`. `local[slot] ± 1` in place. Heap-boxed ints and floats mutate their payload without allocating; small ints re-tag. Traps: `InvalidArgs` on non-numeric.

OP_NIL_LOCAL `<slot:u8>` — 2 bytes. Stack `→`. Releases `local[slot]` and clears it; a slot already empty is left alone. Emitted at a loop back-edge for locals the body declared, so iteration N does not construct its value while iteration N-1 is still referenced from the slot — `OP_SET_LOCAL` releases the outgoing value only once the incoming one exists, which otherwise keeps two generations of a per-iteration structure live at the same time. Clearing is not observable: reading a body local before its declaration yields nil regardless. Traps: —

7.6 Local compound arithmetic

Shared semantics: `aop` is an `ArithOp` (§6.3) with the exact lane rules of §7.2; all forms are zero-stack except `OP_ARITH_L` / `OP_ARITH_IMM8`. Traps: `DivisionByZero` / `ModuloByZero` / `InvalidArgs` from the arithmetic lanes; `RuntimeError` on an out-of-range `aop` that load-validation missed.

OP_ARITH_LC `<dst:u8> <aop:u8> <imm:i8>` — 4 bytes. `local[dst] = local[dst] aop imm`.

OP_ARITH_LL `<dst:u8> <aop:u8> <src:u8>` — 4 bytes. `local[dst] = local[dst] aop local[src]`.

OP_ARITH_L `<dst:u8> <aop:u8>` — 3 bytes. Stack `v →`. `local[dst] = local[dst] aop v`. For `aop = ADD` with a string destination that is uniquely referenced, the append happens **in place** (this is what keeps `s += x` loops linear); an int right operand appends its decimal text without an intermediate string.

OP_ARITH_IMM8 `<aop:u8> <imm:i8>` — 3 bytes. Stack `v → r`. `r = v aop imm`.

OP_ARITH_IMM16 `<aop:u8> <imm:i16>` — 4 bytes. Stack `v → r`. `r = v aop imm`, the 16-bit twin of `OP_ARITH_IMM8` for constants that do not fit a byte. The immediate is sign-extended, so the bitwise operators only ever carry a non-negative mask; a wider constant falls back to a push plus a separate operator.

OP_ARITH_L_IMM8 `<slot:u8> <aop:u8> <imm:i8>` — 4 bytes. Stack `→ r`. `r = local[slot] aop imm` — the `GET_LOCAL`-free form of `ARITH_IMM8`, so it pushes rather than replacing TOS. Same value semantics as the pair it replaces, including int overflow to float and string append; the operand is read from the slot, never consumed, so a non-scalar left operand is copied rather than reused in place.

OP_ARITH_LL_PUSH `<a:u8> <aop:u8> <b:u8>` — 4 bytes. Stack `→ r`. `r = local[a] aop local[b]` — the `ARITH_L_IMM8` twin for two locals instead of an immediate. `aop` is never a comparison (they have no `ARITH_*` selector and keep their own dedicated opcodes) and the compiler never emits this for `+` on two proven strings (stays `OP_CONCAT`); a mixed or unproven `+` still reaches this opcode and follows the same value semantics as the generic path it replaces, including int overflow to float. Both operands are read from their slots, never consumed.

OP_ARITH_FMA_LLL `<dst:u8> <a:u8> <b:u8> <c:u8>` — 5 bytes. Stack `→`. `local[dst] = local[a]·local[b] + local[c]`. The compiler emits it only when `a`, `b`, `c` and the result are all proven ints (`dst = a*b + c`, `dst = c + a*b`, and the accumulate form `dst += a*b`, where `c` is `dst`); a float multiply-add is compiled as a separate multiply and add, so it rounds twice. All-int lane (boxed ints included) wraps in int64 exactly like the separate operators. A float operand from another producer yields the double `a·b + c`; whether that rounds once or twice is unspecified, so a producer SHOULD NOT emit this opcode for non-int operands. All operands are read before `dst` is written, so `c` (or `a`, `b`) may be `dst`. Traps: `TypeMismatch` if any operand is non-numeric; `dst` is left unchanged.

7.7 Property and index access (generic)

OP_GET_PROPERTY `<key:u16>` — 3 bytes. Stack `obj → v`.

- object/instance/exception receiver: hash lookup; a method value is pushed as a **bound method** (receiver captured);
- class receiver: as above, but reading a non-static method raises `ClassNonStaticCall`;
- module receiver: export lookup; a missing export raises `RuntimeError` ("Import error…");
- any other receiver, or a missing property: pushes `nil`, **no raise**.

OP_GET_LOCAL_PROP `<slot:u8> <key:u16>` — 4 bytes. Stack `→ v`. Fused `GET_LOCAL + GET_PROPERTY`: reads `key` from `local[slot]`. Identical to `OP_GET_PROPERTY` in every respect above, including nil-on-missing, except that the receiver is borrowed from the slot rather than popped, so it is never released. Traps: as `OP_GET_PROPERTY`.

OP_SET_PROPERTY `<key:u16>` — 3 bytes. Stack `obj, v →`. Hard-checks: non-object-like receiver → `NullPointer`; `const` receiver → `ConstAssign` (no mutation); **instance receivers are sealed** — the key must be a declared field of the class, else `FieldUndeclared`. Scalars copied on store; the replaced value is released.

OP_GET_INDEX — 1 byte. Stack `c, k → v`. Dispatch on container type:

- array + int: negative index counts from the end; out-of-bounds raises `IndexOutOfBounds`;
- block + int: bounds-checked read of one element (element size 1/2/4/8 bytes) → int, zero-extended for 1/2/4-byte elements, the raw int64 for 8-byte ones;
- object + string: property value or `nil`;
- string + int: one **byte** as a char (negative index from end; OOB raises);
- `nil[k]`: `nil`, no raise;
- any other container: console error, pushes `nil`, **no raise**.

OP_SET_INDEX — 1 byte. Stack `c, k, v →`.

- array + int: a typed array checks the element type (`TypeMismatch`); `k < 0` (after from-end adjust) or `k ≥ 16 777 216` raises `IndexOutOfBounds`; **writing past the end grows the array** to `length = k + 1`;
- block + int: strict bounds (no growth), truncating element store;
- object + string: property store;
- string + int: strict bounds, stores one byte, invalidates the string's cached hash;
- otherwise: `NullPointer`. Const containers: soft `ConstAssign` (§3.4).

7.8 Array fast paths

Fusion hierarchy for reads (fastest first): `GET_ARR_LC` (literal index) → `GET_ARR_LL` (index in local) → `GET_ARR_LI` (int-typed index on TOS) → `GET_INDEX_LOCAL` (any key on TOS) → `GET_INDEX` (fully generic). All are semantically `OP_GET_INDEX` with operands sourced as noted; each falls back to the generic path when the receiver local is not an array, so non-array receivers (string/block/object) still work through them.

`OP_GET_ARR_LC` `<arr:u8> <idx:u8>` — 3 bytes. Stack `→ v`. `v = local[arr][idx]`, `idx` a 0–255 literal.

`OP_GET_ARR_LL` `<arr:u8> <idx:u8>` — 3 bytes. Stack `→ v`. `v = local[arr][local[idx]]`, the index local compiler-proven int. Reads the element inline like `GET_ARR_LC`; a non-array receiver falls through to the generic path exactly as `GET_ARR_LI` does.

`OP_GET_ARR_LI` `<arr:u8>` — 2 bytes. Stack `k → v`. Index popped from TOS (compiler-proven int).

`OP_GET_INDEX_LOCAL` `<arr:u8>` — 2 bytes. Stack `k → v`. Full generic indexing with the container from a local.

`OP_SET_ARR_LC` `<arr:u8> <idx:u8>` — 3 bytes. Stack `v →`. `local[arr][idx] = v` (grows like `OP_SET_INDEX`).

`OP_SET_ARR_LL` `<arr:u8> <idx:u8>` — 3 bytes. Stack `v →`. `local[arr][local[idx]] = v`.

`OP_SET_ARR_LLC` `<arr:u8> <idx:u8> <src:u8>` — 4 bytes. Stack `→`. `local[arr][idx-literal] = local[src]`, zero stack.

`OP_SET_ARR_LLL` `<arr:u8> <idx:u8> <src:u8>` — 4 bytes. Stack `→`. `local[arr][local[idx]] = local[src]`, zero stack.

`OP_SET_INDEX_LOCAL` `<arr:u8>` — 2 bytes. Stack `k, v →`. Generic indexed store, container from a local.

`OP_ARITH_ELC` `<arr:u8> <idx:u8> <aop:u8> <imm:i8>` — 5 bytes. Stack `→`. `local[arr][local[idx]] aop= imm`. Requires an array receiver and int index; **in-bounds only** (never grows). Traps: `InvalidArgs`, `IndexOutOfBounds`, plus arithmetic-lane traps.

`OP_ARITH_ELL` `<arr:u8> <idx:u8> <aop:u8> <src:u8>` — 5 bytes. As `OP_ARITH_ELC` with `local[src]` as the right operand.

`OP_ARITH_EIC` `<arr:u8> <idx:u8> <aop:u8> <imm:i8>` — 5 bytes. Stack `→`. `local[arr][idx-literal] aop= imm`, the `OP_ARITH_ELC` twin for a compile-time literal index instead of a local one. Requires an array receiver; **in-bounds only** (never grows). Traps: `InvalidArgs`, `IndexOutOfBounds`, plus arithmetic-lane traps.

`OP_ARITH_EIL` `<arr:u8> <idx:u8> <aop:u8> <src:u8>` — 5 bytes. As `OP_ARITH_EIC` with `local[src]` as the right operand.

`OP_LOCAL_ARR_LC` `<dst:u8> <arr:u8> <idx:u8>` — 4 bytes. Stack `→`. `local[dst] = local[arr][idx-literal]`, zero stack. Traps: `NullPointer` (non-array), `IndexOutOfBounds`.

`OP_LOCAL_ARR_LL` `<dst:u8> <arr:u8> <idx:u8>` — 4 bytes. Stack `→`. `local[dst] = local[arr][local[idx]]` (array + int index required; negative index from end). Traps: `NullPointer`, `IndexOutOfBounds`.

Two-level indexing `m[i][j]` with all three operands in locals has its own forms; `i` is compiler-proven int as for `GET_ARR_LL`, `j` may be any value. Each behaves exactly as the unfused sequence it replaces - `m[i]`

read as `GET_ARR_LL` reads it, then the one-level read, store or update of that row - with the same traps in the same order. An array of arrays at in-range int indexes is handled inline.

`OP_GET_MAT_LLL` `<m:u8> <i:u8> <j:u8>` — 4 bytes. Stack `→ v`. `v = local[m][local[i]][local[j]]`.

`OP_SET_MAT_LLL` `<m:u8> <i:u8> <j:u8>` — 4 bytes. Stack `v →`. `local[m][local[i]][local[j]] = v`; a store at the row's length grows it, as `OP_SET_INDEX` does. The row is read after `v` was computed, so a compiler emits it only for a `v` whose evaluation can neither raise nor change anything (a local or a literal); anything else keeps the row read first.

`OP_ARITH_MLL` `<m:u8> <i:u8> <j:u8> <aop:u8> <src:u8>` — 6 bytes. Stack `→`. `local[m][local[i]][local[j]] aop= local[src]`. The element is read before it is written, so an out-of-range index traps `IndexOutOfBounds` and never grows the row.

`OP_ARITH_MLC` `<m:u8> <i:u8> <j:u8> <aop:u8> <imm:i8>` — 6 bytes. As `OP_ARITH_MLL` with `imm` as the right operand.

7.9 Object property fast paths

`OP_SET_OBJ_L` `<obj:u8> <key:u16>` — 4 bytes. Stack `v →`. `local[obj].key = v`. Same checks as `OP_SET_PROPERTY`, and `const` is **hard-checked** here (no mutation on `ConstAssign`). Traps: `NullPointer`, `ConstAssign`, `FieldUndeclared`.

`OP_SET_OBJ_LL` `<obj:u8> <key:u16> <src:u8>` — 5 bytes. Stack `→`. `local[obj].key = local[src]`, zero stack; same checks.

`OP_OBJ_ARITH_L` `<obj:u8> <key:u16> <aop:u8> <src:u8>` — 6 bytes. Stack `→`. `local[obj].key aop= local[src]`, zero stack; same checks as `OP_SET_OBJ_LL`. The compound-assign counterpart of that opcode and the non-`this` sibling of `OP_THIS_ARITH_L`. It exists so a string `+=` can append in place: the generic `DUP` / `GET_PROPERTY` / `<op>` / `SET_PROPERTY` sequence reads the field onto the stack, giving it a second reference, which makes an in-place append illegal and the surrounding loop quadratic.

`OP_OBJ_ARITH_S` `<obj:u8> <key:u16> <aop:u8>` — 5 bytes. Stack `v →`. `local[obj].key aop= v`, with `v` popped; same checks and the same in-place string append as `OP_OBJ_ARITH_L`. The field is read after `v` was computed, so a compiler emits it only when the right-hand side has no effects (no call, assignment, `++` / `--`, `await`, `new` or `yield`); any other right-hand side keeps the `DUP` / `GET_PROPERTY` / `<op>` / `SET_PROPERTY` sequence, which reads the field first.

7.10 `this` access

`this` is `local[0]` of a method frame. All `this.*` forms raise `NullPointer` when `this` is nil or absent. The `_PROP` / `_ARITH_L/C` / `_INVOKE` forms are the compiler-emitted, hash-keyed instructions; each self-patches to its slot form on first execution (§3.6).

`OP_GET_THIS_PROP` `<key:u16>` — 3 bytes. Stack `→ v`. On an instance: resolves the member — field → pushes the field (patches to `OP_GET_THIS_SLOT`); class const → pushes the shared value (patches to `OP_GET_THIS_CONST`); method → pushes a bound method; missing → `nil`. On a plain-object `this`: hash/proto-chain lookup, methods bound, missing → nil.

`OP_SET_THIS_PROP` `<key:u16>` — 3 bytes. Stack `v →`. Instance: key must be a declared field (`FieldUndeclared`), patches to `OP_SET_THIS_SLOT`, stores into the field slot. Plain object: property store.

Const `this` : soft `ConstAssign` .

`OP_THIS_ARITH_L` `<key:u16> <aop:u8> <src:u8>` — 5 bytes. Stack `→` . `this.key aop= local[src]` . Patches to `OP_THIS_ARITH_SLOT_L` . Traps: `NullPointer` , `FieldUndeclared` , arithmetic-lane traps.

`OP_THIS_ARITH_C` `<key:u16> <aop:u8> <imm:i8>` — 5 bytes. As above with an i8 immediate; patches to `OP_THIS_ARITH_SLOT_C` .

`OP_GET_THIS_SLOT` `<slot:u16>` — 3 bytes. Stack `→ v` . Pushes instance field `slot` directly. The operand is a resolved field slot, **not** a constant index; guarded at runtime (`this` must be a real instance and `slot < fieldCount` , else `NullPointer` / `RuntimeError`).

`OP_SET_THIS_SLOT` `<slot:u16>` — 3 bytes. Stack `v →` . Stores into the field slot (same guard; soft `ConstAssign` on a const instance).

`OP_GET_THIS_CONST` `<memberIdx:u16>` — 3 bytes. Stack `→ v` . Pushes the shared class-const member. On a guard failure the class layout is rebuilt once before raising (`NullPointer` / `RuntimeError`).

`OP_THIS_ARITH_SLOT_L` `<slot:u16> <aop:u8> <src:u8>` — 5 bytes and **`OP_THIS_ARITH_SLOT_C`** `<slot:u16> <aop:u8> <imm:i8>` — 5 bytes. Compound assign directly on the field slot; same guard, same arithmetic lanes.

`OP_BIND_THIS` — 1 byte. Stack `f → bm` . Pushes a bound method binding the current `this` (`local[0]`) to `f` . Used for anonymous functions declared in method bodies. Traps: —

`OP_CALL_WITH_THIS` `<argc:u8>` — 2 bytes. Stack `recv, a₁ … a_argc, f → result` . Calls the function value `f` with `recv` bound to `local[0]` , the way an instance method is called - the callee sees `recv` as `this` and the arguments in `local[1..argc]` . Unlike the method-invocation opcodes, the callee here is a value already on the stack, not a member looked up on the receiver. Emitted for the init block of `new C(...) { ... }` (§ the compiler compiles the block as an anonymous thiscall function). Traps: `StackError` (fewer than `argc + 2` values on the stack), `TypeMismatch` (`f` is not a function), `InvalidArgs` (arity), plus whatever the callee raises.

7.11 Construction

`OP_BUILD_OBJECT` `<count:u16> <keys:const:u16>` — 5 bytes. Stack `v₁, … v_n → obj` . Pops `count` values and builds a fresh object, storing `vᵢ` under the string constant at index `keys + i − 1` : the keys are `count` consecutive pool entries, inserted in order. The object holds those constants themselves as its keys; string constants are immutable, so a write into one raises `ConstAssign` . Scalar values copied. Traps: `StackError` (underflow).

`OP_BUILD_ARRAY` `<count:u16> <elemType:type:u16>` — 5 bytes. Stack `v₁, …, v_n → arr` . Consumes `count` values; the deepest is element 0 and TOS is element `count−1` , so the compiler pushes `[a, b, c]` in source order. `elemType` = 0 → untyped array; `elemType = VAL_FLOAT` → flat float storage (each element converted to a double; a non-numeric element becomes 0.0, `true` 1.0); `elemType = VAL_INT` → flat int storage (a non-int element raises `TypeMismatch`); `elemType = VAL_MATRIX | T` → a `[[T]]` matrix, each element an array of exactly type `VAL_ARRAY | T` (anything else raises `TypeMismatch`); any other non-zero `elemType` → typed array — a value whose type doesn't intersect the mask raises `TypeMismatch` . Traps: `StackError` , `TypeMismatch` .

`OP_MAKE_CONST` — 1 byte. Stack `v → v`. Marks the (heap) value on TOS immutable in place; immediates are already immutable. Traps: —

`OP_NEW` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n, class → inst`. Pops the class (else `RuntimeError`), links its inheritance chain, creates a slot-backed instance with every declared field initialized from its default (deep-cloned), then runs the constructor if the class chain has one: a bytecode constructor runs in a frame prepared directly on the instance (no bound-method wrapper, so the instance's lifetime ends with its last script reference) with the given args and its return value discarded; a native constructor is called directly. A class without a constructor drops the arguments. Arity mismatch raises `InvalidArgs`. The instance is pushed. Traps: `RuntimeError`, `InvalidArgs`, plus anything the constructor raises.

`OP_SUPER` — 1 byte. Stack `→ bm`. Pushes `this` bound to the `Constructor` of the **base of the defining class** (`frame->owner`'s proto) — the explicit `super(...)` call then goes through `OP_CALL`. Traps: `NullPointer` (no usable `this`, no owner, no base, or base has no constructor).

`OP_GUARD` — 1 byte. Stack `v → v` (kept) or `v →` + raise. If TOS is nil: pops it and raises `GuardNull`; otherwise leaves the value. Compiled from guard/`!` expressions. Traps: `GuardNull`.

7.12 Calls and returns

Call-frame mechanics, arity and typed-parameter rules: §3.2.

`OP_CALL` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n, f → r`. Calls `f`. Plain function → new frame (args become `local[0..argc-1]`, omitted optionals nil-filled). Bound method → receiver becomes `local[0]`, args shift up. Builtin → executed inline, argument types validated. Class → constructor sugar. Async function → spawns a fiber and pushes the **fiber object** immediately (not scheduled until awaited/run). Non-callable → console error, pushes nil, **no raise**. Traps: `StackError` (frame budget), `TypeMismatch` (empty callee slot), `InvalidArgs` (arity/types).

`OP_CALL_SELF` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n → r`. Self-recursive call: the callee is the current frame's function — no lookup, no callee on the stack. The compiler emits it only where every argument provably fits its declared parameter type, and the VM does not check the types again: the claim reaches no function but the one that carries it. Traps: `RuntimeError` (no current function), `StackError`, `InvalidArgs` (argument count).

`OP_TAIL_SELF` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n →` (frame restarts). Tail self-call: rebinds the current frame's locals to the new arguments and restarts at ip 0. Constant stack and frame depth. As with `OP_CALL_SELF`, the argument types are not checked again. Traps: — (argument shapes are the same function's).

`OP_TAIL_CALL` `<argc:u8>` — 2 bytes. Stack `arg₁ … arg_n, f → r`. Fused call+return. A plain function callee **replaces** the current frame (true tail-call elimination — constant frame depth); one with native code runs natively and its result is returned from the current frame. Any other callee degrades to call-then-return. Traps: as `OP_CALL`.

`OP_RETURN` — 1 byte. Stack (callee) `r →` / (caller) `→ r`. Returns TOS to the caller; tears down the frame (§3.2). Returning from the last frame finishes the fiber (§3.3).

`OP_RETURN_NONE` — 1 byte. Returns **no value**; used as the terminator of top-level/module chunks (function bodies always return a value).

`OP_RETURN_NIL` — 1 byte. Pushes nil, then behaves as `OP_RETURN`. The implicit function epilogue.

`OP_RETURN_L` `<slot:u8>` — 2 bytes. Returns `local[slot]` (fused `GET_LOCAL + RETURN`).

`OP_FN` `<fnIdx:u16>` — 3 bytes. Stack `→ f`. Pushes the function constant after re-pointing its environment at the current environment. No capture, no clone. The compiler emits it only for a function declared at **module scope**, where that environment outlives the program; a literal inside a function body gets `OP_FN_CAPTURE` even with nothing to capture, so it is a clone with an environment of its own. Writing a closure's environment into the shared constant would otherwise publish it to every later evaluation of the same literal, and leave it dangling once that closure is released. Traps: `RuntimeError` (constant is not a function).

`OP_FN_CAPTURE` `<fnIdx:u16> <count:u8> count×(<name:u16> <parentSlot:u8>)` — 4 + 3·count bytes. Stack `→ f`. `count` may be 0, which still clones and still creates the child environment. Closure creation: clones the function template, creates a child environment of the current one, and snapshots each captured local (`local[parentSlot]`) into it under the given name. Captures are **by value-reference, at creation time**: rebinding the outer local later is invisible to the closure; mutating a captured container is visible. Inside the body, captured names resolve via `OP_GET_GLOBAL` through the closure environment. Traps: `RuntimeError`.

`OP_YIELD` — 1 byte. Stack `v →`. Suspends the fiber. If a resumer/awaiter is attached, `v` is delivered as its resume value; otherwise `v` is discarded. An awaited generator is re-queued; an un-awaited one parks. Traps: —

`OP_AWAIT` — 1 byte. Stack `v → r`. `v = nil`: plain suspension (resumes with whatever is delivered). `v` a finished fiber: pushes its preserved result immediately, no suspension. `v` a live fiber: parks this fiber until the target completes; the target's result becomes `r` (a fiber that dies without a result delivers nil). Traps: `InvalidArgs` (non-nil non-fiber operand).

7.13 Method invocation

Several opcodes below share one **generic resolution** for a receiver whose exact kind isn't (or can't be) index-cached: object-likes by hash (plain objects also fall back to the `Object.*` builtin namespace); modules by export; string/char/array receivers sugar into the `String.*` / `Char.*` / `Array.*` builtin with the receiver as first argument. **A missing method drains the args and pushes nil — no raise.** A property that holds a non-function: console error + nil. Builtin methods validate arity and types (`InvalidArgs`). Traps: `RuntimeError` (module without exports), `InvalidArgs`, plus callee raises. Each opcode below falls back to this generic resolution whenever its own inline cache doesn't apply.

`OP_THIS_INVOKE` `<key:u16> <argc:u8>` — 4 bytes. Stack `arg₁ … arg_n → r`. `this.method(args)`; resolves on the receiver's class chain, honors overrides, self-patches to `OP_THIS_INVOKE_SLOT` when the name resolves to a method of the defining class. Traps: `NullPointer` (unusable `this`), plus the generic resolution above on fallback.

`OP_THIS_INVOKE_SLOT` `<memberIdx:u16> <argc:u8>` — 4 bytes. Patched vtable form: `memberIdx` indexes the receiver class's base-first member layout; the member reference is re-read every call, so subclass overrides and hot-patched functions take effect without re-patching. Guard failure rebuilds the layout once, then raises (`NullPointer` / `RuntimeError`).

`OP_TAIL_THIS_INVOKE` `<key:u16> <argc:u8>` — 4 bytes, same shape as `OP_THIS_INVOKE` (a compiler tail-call pass patches the opcode byte in place when a `this.method(args)` call is immediately followed by `return`). Same resolution and self-patch as `OP_THIS_INVOKE` (into `OP_TAIL_THIS_INVOKE_SLOT`, not

`OP_THIS_INVOKE_SLOT`), but a call to a plain, non-async script function reuses the current frame instead of pushing a new one - `this` stays exactly where it is at `locals[0]`, since a `this.method()` tail call always calls with the same receiver, so only the callee's code and this call's own arguments change. Anything not eligible for reuse (an async method, or resolution falling through to the generic mechanism) falls back to exactly `OP_THIS_INVOKE`'s own behavior - a real frame push, correct but without the tail-call stack benefit.

`OP_TAIL_THIS_INVOKE_SLOT` `<memberIdx:u16> <argc:u8>` — 4 bytes. Tail-position counterpart of `OP_THIS_INVOKE_SLOT`: same patched-vtable resolution, same frame-reuse eligibility split as `OP_TAIL_THIS_INVOKE`.

`OP_INVOKE_INSTANCE` `<key:u16> <argc:u8> <cache:u16>` — 6 bytes. Stack `arg₁ … arg_n, obj → r`. `obj.method(args)` specialized for receivers the analyzer proved to be instances (else `RuntimeError`). The trailing u16 is an **inline cache**, rewritten at runtime: a cached layout index is trusted only after its name-hash matches, so call sites shared by different classes stay correct. Falls back to the generic resolution above when the cache misses and the name isn't a class method.

`OP_INVOKE_GLOBAL_CACHED` `<globalKey:u16> <methodKey:u16> <argc:u8> <mode:u8> <classIdGuard:u32> <idx:u16>` — 13 bytes. Stack `arg₁ … arg_n → r`. Fused `GET_GLOBAL` + `INVOKE_MEMBER` for `Identifier.method(args)` on a global receiver (imported module, class statics); the receiver never touches the operand stack. `globalKey` resolves through a plain hashmap lookup every call (already O(1), not cached here). The method resolution self-classifies exactly like `OP_INVOKE_MEMBER`'s class-receiver case: once `globalKey` resolves to a class whose member is a shared method or const, `mode 1` caches the class's member-layout index, trusted only while `classIdGuard` matches. A module receiver, a field member, or a lookup miss is never cached (`mode` stays `0`) - falls back to the generic resolution above.

`OP_INVOKE_MEMBER` `<key:u16> <argc:u8> <mode:u8> <classIdGuard:u32> <idx:u16>` — 11 bytes. Stack `arg₁ … arg_n, obj → r`. `obj.method(args)` where the receiver's exact kind isn't proven. Self-classifying: `mode 0` always re-derives the receiver's kind. A plain object never patches past `mode 0` (no fixed slot exists to cache against — falls back to the generic resolution above on every call). A class or instance receiver patches to `mode 1` (member is a shared method/const: `idx` indexes the class's member layout, re-read every call) or `mode 2` (member is a field holding a callable: `idx` is the field slot, its value re-read and re-checked every call). Either mode is trusted only while `classIdGuard` matches the receiver's class; a mismatch reclassifies, falling back to the generic resolution above when the name isn't a class method.

`OP_SUPER_INVOKE` `<key:u16> <argc:u8>` — 4 bytes. Stack `arg₁ … arg_n → r`. `super.method(args)`: resolves on the **base of the defining class** (never the receiver's dynamic class — this is what makes an overriding method able to call the overridden one without recursing). Self-patches to `OP_SUPER_INVOKE_SLOT` on a method hit. Traps: `NullPointer` (unusable `this`/owner/base), `RuntimeError` (method not found on the base — no dynamic fallback).

`OP_SUPER_INVOKE_SLOT` `<memberIdx:u16> <argc:u8>` — 4 bytes. Patched form: since the base class is fixed per call site, `memberIdx` indexes its member layout directly with no classId guard needed. Ref re-read every call.

`OP_CALL_GLOBAL` `<globalKey:u16> <argc:u8> <pad:u8>` — 5 bytes. Stack `arg₁ … arg_n → r`. Fused `GET_GLOBAL + CALL`; the callee is resolved from the environment and never pushed. The pad byte keeps the fused instruction the length of the pair it replaces; it holds the value of `OP_NOP`, is skipped and is never executed. Produced by the peephole pass **after** tail-call optimization, so tail sites keep TCO. An undefined global resolves to nil → "call to non-callable" console error + nil result. Traps: as `OP_CALL`.

7.14 Builtin call shortcuts

`OP_PUSH_BUILTIN` `<mod:u8> <fn:u8>` — 3 bytes. Stack `→ f`. Pushes builtin function `fn` of builtin module `mod` (indices into the builtin registry, validated at load). Traps: —

`OP_CALL0_BUILTIN` … `OP_CALL5_BUILTIN` `<mod:u8> <fn:u8>` — 3 bytes. Stack `arg₁ … arg_N → r` (N = 0…5). Fused builtin call: pops N args, validates arity and declared argument types (`InvalidArgs`; nil passes any type), runs the builtin, pushes its result — a void builtin pushes nothing (the compiler accounts for this statically). Equivalent to `PUSH_BUILTIN + CALL N` without the callee round-trip. Traps: `InvalidArgs`, plus whatever the builtin raises.

7.14a Host call shortcuts

A host embedding the VM can publish native modules of its own (in its host options, or with `FlarisRegisterModule` afterwards). Those modules are not in the builtin registry and have no index a chunk could carry — each host keeps its own registry, and its ordering depends on the host, not on the program. A host call therefore carries the two 32-bit name hashes instead: `xxHash64(moduleName)` and `xxHash64(functionName)`, each truncated to its low 32 bits.

`OP_PUSH_HOST` `<modHash:u32> <fnHash:u32>` — 9 bytes. Stack `→ f`. Pushes the host function the two hashes name. Traps: —

`OP_CALL0_HOST` … `OP_CALL5_HOST` `<modHash:u32> <fnHash:u32>` — 9 bytes. Stack `arg₁ … arg_N → r` (N = 0…5). As the builtin forms: pops N args, validates declared argument types, runs the host function, pushes any result. Arity is checked at link time rather than per call. Traps: `InvalidArgs`, plus whatever the host function raises.

Linking. Both hashes are resolved once, when the chunk is loaded, and the resolved member pointer is written over the 8-byte payload in place — so the executing form is a single load with no table indirection and no bounds check. A hash that resolves to nothing rejects the chunk; the error lists the modules and functions the host *did* register, because a hash cannot be reversed to the name the source wrote. Registration refuses two names whose 32-bit hashes collide, so a collision is a startup error rather than a silently wrong call target.

Two consequences follow from the in-place rewrite:

- A **linked chunk must never be serialised** — its payloads are pointers, not hashes. Nothing writes a chunk back out today, and `chunk->hostLinked` marks the ones that must not start.
- A `.flx` containing host calls is bound to a host that registers those exact names. That is deliberate: these chunks are a host's own scripts, compiled at load, not artifacts meant to run anywhere.

7.15 Exception handling

The state machine these instructions drive is specified in §3.4.

`OP_TRY_BEGIN` `<catchIp:addr:u16> <catchSlot:u8> <finallyIp:addr:u16>` — 6 bytes. Stack `→`. Pushes a try context (state captured: frame count, stack depth, live locals). `catchIp` / `finallyIp` are absolute; `catchSlot` is where a caught exception will bind. Traps: `RuntimeError` (> 24 nested contexts).

`OP_TRY_END` — 1 byte. Normal try-body completion: jumps to the finally landing pad. Traps: `RuntimeError` (no try context in this frame — crafted bytecode).

OP_CATCH_BEGIN — 1 byte. Stack `e →`. Binds the delivered exception (pushed by the delivery machinery) into `local[catchSlot]`. Traps: `RuntimeError` (no active context — crafted bytecode).

OP_CATCH_END — 1 byte. Normal catch completion: jumps to the finally pad. Traps: `RuntimeError` (no context).

OP_FINALLY_BEGIN — 1 byte. Marks the context in-finally. Traps: `RuntimeError` (no context).

OP_FINALLY_END — 1 byte. Pops the context and resumes its pending action: none → fall through; raise → re-raise the saved exception; return/break/continue → chain outward through enclosing finallys, then return the saved value / jump to the saved target (§3.4). Traps: — (re-raise aside).

OP_TRY_LEAVE `<kind:u8> <count:u8> <target:addr:u16>` — 5 bytes. Stack: kind 0 pops the return value; kinds 1/2 pop nothing. Records a pending `return` (0), `break` (1) or `continue` (2) that must first run `count` levels of enclosing finally bodies, then jumps to the innermost finally. `target` is the break/continue destination. Executed inside a finally body (crafted bytecode; the compiler never emits it there): `RuntimeError`.

OP_THROW — 1 byte. Stack `e →`. Raises `e`, which MUST be an instance whose class chain reaches the builtin `Exception` class, else `TypeMismatch`. Delivery per §3.4; uncaught → process exit.

7.16 Iteration

OP_FOREACH_BEGIN `<valueSlot:u8> <keySlot:u8> <iterSlot:u8> <indexSlot:u8> <exit:addr:u16>` — 7 bytes. Stack `→`. The first iteration step of a `foreach` loop, run once before the body. `local[indexSlot]` holds the cursor, which starts before the first element whatever the slot held (`OP_FOREACH_NEXT` continues from it). Only `keySlot` may be `0xFF` ("the loop declares no key variable"); the value, iterable and index slots are read on every step and the validator requires them to be real slots. Per container in `local[iterSlot]`:

- array: advance index; exhausted when `index ≥ length`; value = element, key = index;
- string: as array over **bytes**; value = char;
- object / instance / exception / class: advance an opaque map cursor (iteration order is insertion order — a key deleted and added again moves to the end; structural mutation during iteration is undefined); value = entry value, key = entry key; a slot-backed instance with no dynamic map is immediately exhausted;
- other: raise `InvalidArgs`. On exhaustion: value/key slots are set to nil and control jumps to the absolute `exit` address; otherwise the slots are filled and control falls through into the loop body.

OP_FOREACH_NEXT `<valueSlot:u8> <keySlot:u8> <iterSlot:u8> <indexSlot:u8> <off:u16>` — 7 bytes. Stack `→`. The loop's back-edge, after the body: the same step from the cursor in `local[indexSlot]`. When the slots are filled, control jumps back to `base − off`, the start of the body, a scheduling checkpoint as at `OP_LOOP`; on exhaustion the value/key slots are set to nil and control falls through.

OP_ITER_BEGIN `<slot:u8> <endSlot:u8> <exit:addr:u16>` — 5 bytes. Stack `→`. Runs once per counted `iter` loop. Both `local[slot]` (start) and `local[endSlot]` (end) MUST be ints, else `InvalidArgs`; either may be heap-boxed, so the endpoints are not limited to the tagged-int range. If `start >= end`, jumps to `exit` (empty range - the source range's upper bound is exclusive, so `start == end` is already empty). Otherwise, a span wider than `MAX_ITER_SPAN` (2^{60} steps) raises `InvalidArgs` — the *width* is refused, never the magnitude, so a short range at any magnitude is legal.

OP_ITER_NEXT `<slot:u8> <endSlot:u8> <off:u16>` — 5 bytes. Stack `→`. If `local[slot] < local[endSlot]`: increments the slot and branches back `off` bytes to the body, like `OP_LOOP` (quantum-checked back-edge); otherwise falls through. The test is inclusive against the STORED bound, which is one less than the source range's upper bound — a counted loop decrements it once at entry (§ `OP_DEC_LOCAL`), so the language's exclusive `to` costs nothing per iteration. When both bounds are tagged ints the increment is a pure immediate rewrite; if either is boxed, the step goes through a boxed lane that releases the outgoing counter. Traps: —

7.17 Type operations

All are 1 byte, stack `v → r`.

OP_LEN — `r:int` = string byte length / array length / block element count / object property count / instance declared-field count / class method+const count; every other type → 0. Traps: —

OP_TYPE — `r:int` = the value's `ObjectType` bit (§4.1). Traps: —

OP_HAS_KEY — Stack `k, c → r:bool` (key below, container on top). object: property presence. array: deep-equality membership test. string container: containment - a **string** key is a byte-substring test (the empty string is contained in every string), a **char** key is encoded to UTF-8 and searched the same way. Other combinations → false. Traps: —

OP_TO_STRING — canonical text: strings pass through; int decimal; float `%.17g`-equivalent; `true` / `false`; char as quoted UTF-8; `nil` → `"nil"`; containers/functions → `<array[N]>`, `<object>`, `<fn>`, builtin name, `<module>`, `<exception>`; block → its raw bytes as a string. Traps: —

OP_TO_INT — int identity; float truncates toward zero; string parsed as decimal (garbage → 0); bool/char → value; nil → 0; other → `InvalidArgs`.

OP_TO_FLOAT — analogous (`string` via decimal float parse). Traps: `InvalidArgs`.

OP_TO_CHAR — char/int/bool → codepoint; string → first byte (empty → 0); nil → 0; other → `InvalidArgs`.

OP_TO_I8 / **OP_TO_I16** / **OP_TO_I32** — coerce to int as `OP_TO_INT` does, then sign-extend the low 8/16/32 bits. **OP_TO_U8** / **OP_TO_U16** / **OP_TO_U32** — coerce to int, then zero-extend the low bits. Traps: none; a value `OP_TO_INT` rejects coerces to 0.

OP_CHECK_TYPE `<mask:u32> <label:u16>` — 7 bytes, stack `v → v'`. The value on top is about to be stored in a variable, or returned from a function, whose declared type is `mask` (§4.1 bits; a typed array keeps its element bits). The compiler emits it where the value's type is not known statically, and where a known int is bound for a mask that admits float but not int, or a known char for one that admits int but not char (a literal is converted at compile time instead). `v` fits when it shares a type bit with `mask`, with these rules: nil fits a mask holding `VAL_NIL` or any reference type; any array fits an array mask, and nothing else does; an int is converted to float when the mask admits float but not int, and a char to int when it admits int but not char (`v'` is the converted value). `label` indexes a string constant naming the slot for the error message. Traps: `TypeMismatch` (22), `v` released.

7.18 Modules

OP_EXPORT `<aliasIdx:u16> <nameIdx:u16> <type:u32>` — 9 bytes. Stack `→`. Binds export `alias` to the value of the binding `name` in the module's **own** environment (imported/builtin names from parent

environments cannot be re-exported). `type` is the exported symbol's declared `ValMask`, recorded so importers can read export types statically from the `.flx`; the VM ignores it at run time. Traps: `RuntimeError` (missing local binding, non-string constants).

7.19 Debug

All are semantically transparent. Only `OP_DBG_LINE` depends on debug info (§5.5); the other two are emitted in every chunk.

`OP_DBG_LINE` `<line:u16>` — 3 bytes. Sets the frame's current source line (stack traces, debugger line stepping). Emitted where the source line changes, not per statement: two statements on one line share one marker. Lines above 65,535 saturate to 65,535 (the compiler warns once). A frame that has executed no `OP_DBG_LINE` reports line 0. Traps: —

`OP_DBG_FILE_NAME` `<idx:const:u16>` — 3 bytes. Sets the chunk's source-file name. Traps: —

`OP_DBG_BREAK` `<code:u16>` — 3 bytes. Debugger breakpoint: interactive break in the full build, an informational print in the runtime-only build. Traps: —

7.20 Block (raw memory) fast paths

All ten require `local[blk]` to be a block, else `TypeMismatch`. Reads zero-extend the block's element (element size 1/2/4 bytes; an 8-byte element is the raw int64; other sizes → `RuntimeError`) and push an int; writes truncate an int to the element size. Negative indices count from the end. Out-of-bounds raises `IndexOutOfBounds`. The payload is VM-owned memory bounded by the block's own length, so no further address check is made.

`OP_GET_BLK_LL` `<blk:u8> <idx:u8>` — 3 bytes. Push `blk[local[idx]]`. **`OP_GET_BLK_LI`** `<blk:u8>` — 2 bytes. Stack `k → v`. Index popped from TOS (compiler-proven int), the `OP_GET_BLK_LL` twin for a computed index that isn't a bare local, e.g. `buf[i*2]`. Unlike `OP_GET_ARR_LI`, a non-block receiver is not a generic-path fallback - the compiler only ever emits this for a proven-block receiver, so a mismatch here means corrupt bytecode and raises `TypeMismatch` instead. **`OP_SET_BLK_LL`** `<blk:u8> <idx:u8>` — 3 bytes. Stack `v →`. `blk[local[idx]] = v`. **`OP_SET_BLK_LLL`** `<blk:u8> <idx:u8> <src:u8>` — 4 bytes. Zero-stack `blk[local[idx]] = local[src]`.

7.21 String concatenation

`OP_CONCAT` — 1 byte. Stack `a, b → r`. Binary string concatenation: both operands are coerced to text (chars UTF-8-encoded, numbers formatted) and joined into a fresh string. Emitted only when the compiler proves both operands are strings; semantically equal to `OP_ADD`'s string lane without the arithmetic dispatch. Traps: —

`OP_CONCAT_N` `<n:u8>` — 2 bytes. Stack `v₁ … v_n → r`. N-ary string concatenation: pops `n` operands (deepest = leftmost), coerces each as `OP_CONCAT` does, and joins them in one pass — one allocation, one hash, each piece copied exactly once (a binary cascade recopies every prefix). `n` MUST be 2…16 (`MAX_CONCAT_FUSE`); the loader rejects other values. Emitted for `a + b + c` chains whose leaves are all statically string/char; longer chains are compiled as nested fusions. Semantically equal to a left-associative cascade of `OP_CONCAT`. Traps: —

---

8. Loader validation

Validation is **fail-closed**: anything not explicitly modeled is rejected. The reference loader reports a rejection to its caller: the command-line VM exits with code 4 (`EXIT_CODE_ERR_FILE`), an embedding host receives a failed load status. Malformed bytecode is never surfaced as a catchable runtime exception. Two deliberate exceptions to "reject": a malformed **bundled dependency** is skipped with an error (the outer program still loads), and a malformed **JIT IR section** is dropped (the function runs interpreted).

8.1 File level

1. File size ≥ 192; header readable in full.
2. `magic == "FLS2"` .
3. `MIN_SUPPORTED_BYTECODE_VERSION ≤ compilerVersion ≤ SYS_VERSION` .
4. Signature policy (§5.3): `BAD` / `STRIPPED` / `UNSUPPORTED` always rejected; `UNSIGNED` / `VALID` rejected under `--require-signed` .
5. `mainSectionStart ≥ 192` ; non-bundle: `mainSectionStart == 192` .
6. Section ordering `mainSectionStart ≤ stringPoolStart ≤ stringPoolEnd ≤ codeStart ≤ codeEnd ≤ mainSectionEnd ≤ fileSize` .
7. Main-section length ≥ 1 and ≤ `MAX_CODE_SIZE` (10 MiB).
8. Every read is bounds-checked overflow-safely (compare `n > size - pos` , never `pos + n > size`); a short read is fatal.
9. `flags` has no bit set other than `FLAG_BUNDLE` ; the reserved bytes at offsets 80, 83 and 180 are zero.
10. Bundles: `mainSectionStart == 194 + entryCount·152` , so the main section follows the TOC directly.

8.2 Deserialization level

- String pool: entry `len < MAX_STRING_SIZE` ; reference `slot < count` .
- Chunk: nesting depth < 64 (§5.6); chunk magic `0xA5` ; version window as above; `consts ≤ 16384` ; `1 ≤ codelen ≤ 10 MiB` ; debug `localCount ≤ 128` ; debug-name and IR lengths within their caps.
- Values: depth < 64; string/array/object/class size caps (§5.6); class member `kind ≤ 2` ; object keys, class names and member names must be strings; unknown type tag fatal. Function `arity` / `localCount` are clamped (not rejected) and function flags sanitized (§5.6).
- Every declared length or count must also fit in the bytes that remain in the buffer, checked before any allocation is sized from it.

8.3 Chunk validation

Every chunk — freshly compiled or deserialized — MUST pass the loader's validator before execution; it recurses into every function constant's sub-chunk (recursion cap 256). It rejects on: NULL/empty code, a constant table announced but absent, and **string-constant hash collisions** — two string constants whose 64-bit hashes are equal but whose bytes differ (the hash is the sole runtime identity of names, §4.6; identical-byte duplicates are legal).

Pass 1 — instruction decode. Every instruction in `[0, codelen)` is decoded. Rejected: unknown opcode; truncated operands; constant index ≥ `constCount` ; key operands whose constant is not a real heap string (for `OP_BUILD_OBJECT` , each of its `count` keys); local slot ≥ 128 — including each `OP_FN_CAPTURE` `parentSlot` and the `OP_FOREACH_BEGIN` / `OP_FOREACH_NEXT` value, iterable and index slots, of which only the key slot may be the sentinel `0xFF` (the value, iterable and index slots are indexed unconditionally and must be real); argc > 16 on every call/invoke form; `OP_CONCAT_N` count outside 2…16 (`MAX_CONCAT_FUSE`); arithmetic sub-op ≥ 12;

relative branch target outside `[0, codelen]`; absolute target ≥ `codelen`; `OP_TRY_BEGIN` catchSlot ≥ 128; `OP_JUMP_TABLE` with `max < min`; builtin references out of the builtin registry's range, and `OP_CALL0..5_BUILTIN` targets that are not builtin functions; `OP_EXPORT` indices out of range; an `OP_CHECK_TYPE` label index ≥ `constCount`. A host-call opcode (`OP_PUSH_HOST`, `OP_CALL0..5_HOST`) present in the code makes the chunk require host linking, so a crafted chunk cannot leave one unlinked for the interpreter to dereference: linking either binds every host call to a registered member or rejects the chunk. The bytes consumed per opcode are cross-checked against the canonical length table (§6.5); any drift is a reject.

Pass 2 — jump edges. Every branch target collected in pass 1 must land on an instruction **boundary** recorded in pass 1 (jumping into the middle of an instruction is rejected).

Pass 3 — abstract stack simulation. The validator symbolically executes the code, tracking operand-stack depth along all paths:

- Every reachable instruction has **one** depth. All edges into it - fall-through, forward and backward branches, absolute targets, the catch and finally entries of a try, a tail call's fall-through - must agree, or the chunk is rejected. A loop body therefore leaves the stack as it found it, and `maxStack` bounds every path and every iteration.
- An instruction that takes more values than the depth it runs at (the values it pops plus any it reads below the top) is rejected - never clamped. This holds for terminating instructions too.
- Some instructions must run at an exact depth, so nothing below their operands survives: `RETURN` at 1; `RETURN_NIL`, `RETURN_L` and `RETURN_NONE` at 0 (`RETURN_NONE` only at top level); `TAIL_SELF` and `TAIL_THIS_INVOKE` at `argc`; `TAIL_CALL` at `argc + 1`; `TRY_BEGIN`, `TRY_END`, `CATCH_END` and `FINALLY_END` at 0; `TRY_LEAVE` at 0 once its value is taken.
- Together with the per-frame reservation, these rules let the interpreter push and pop without bounds checks.
- Code that can fall off the end of the chunk without a terminating instruction (`RETURN*`, `TAIL_*`, `THROW`, `JUMP`, `LOOP`, `JUMP_TABLE`, `TRY_LEAVE`) is rejected.
- Any opcode pass 1 accepts but pass 3 does not model is rejected (fail closed).
- The maximum depth reached is recorded as the chunk's `maxStack` (clamped to 65535); the VM trusts it to reserve stack space once per call instead of checking per push.

Per-instruction depth deltas (net effect; branch seeds in parentheses):

Delta	Instructions
+1	`NIL` `TRUE` `FALSE` `DUP` `SUPER` `CONSTANT` `CONSTANT_REF` `IMM8/16/32` `IMM_CHAR` `GET_LOCAL` `GET_LOCAL0-3` `GET_GLOBAL` `GET_THIS_PROP` `GET_THIS_SLOT` `GET_THIS_CONST` `FN` `FN_CAPTURE` `GET_ARR_LC/LL` `GET_LOCAL_PROP` `GET_BLK_LL` `PUSH_BUILTIN` `CALL0_BUILTIN` `PUSH_HOST` `CALL0_HOST` `ARITH_L_IMM8` `ARITH_LL_PUSH` `GET_MAT_LLL`
0	unary ops, `TO_*` converters, `GET_PROPERTY` `GET_ARR_LI` `GET_BLK_LI` `GET_INDEX_LOCAL` `ARITH_IMM8/IMM16/LC/LL/ELC/ELL/EIC/EIL` `OBJ_ARITH_L` `INC/DEC_LOCAL` `NIL_LOCAL` `SET_ARR_LLC/LLL` `SET_OBJ_LL` `THIS_ARITH_*` `LOCAL_ARR_*` `SET_BLK_LLL` `DBG_*` `EXPORT` `CHECK_TYPE` `CALL1_BUILTIN` `CALL1_HOST` `ARITH_FMA_LLL` `CATCH_END` `FINALLY_BEGIN/END` `TRY_END` `MAKE_CONST` `GUARD` `BIND_THIS` `AWAIT` `CMP_C` `NOP` `ARITH_MLL/MLC`
−1	`POP`, binary arithmetic and comparisons, `HAS_KEY` `GET_INDEX` `CONCAT` `SET_LOCAL` `SET_LOCAL0-3` `SET_GLOBAL` `DEFINE_GLOBAL` `SET_ARR_LC/LL` `SET_OBJ_L` `SET_THIS_PROP` `SET_THIS_SLOT` `SET_BLK_LL` `CALL2_BUILTIN` `CALL2_HOST` `ARITH_L` `CATCH_BEGIN` `YIELD` `SET_MAT_LLL`
−2	`SET_PROPERTY` `SET_INDEX_LOCAL` `CALL3_BUILTIN` `CALL3_HOST`
−3	`SET_INDEX` `CALL4_BUILTIN` `CALL4_HOST`

Delta	Instructions
−4	`CALL5_BUILTIN` `CALL5_HOST`
−argc	`CALL` `NEW` `INVOKE_INSTANCE` `INVOKE_MEMBER` (callee/receiver replaced by result)
1−argc	`INVOKE_GLOBAL_CACHED` `THIS_INVOKE` `THIS_INVOKE_SLOT` `SUPER_INVOKE` `SUPER_INVOKE_SLOT` `CALL_GLOBAL` `CALL_SELF` (receiver resolved internally, result pushed)
−1−argc	`CALL_WITH_THIS` (receiver, args and callee all popped, result pushed)
1−count	`BUILD_OBJECT` `BUILD_ARRAY`
1−n	`CONCAT_N`

Branch seeding: `JUMP_IF_FALSE/TRUE` continue at `d−1` and seed their target at `d−1`; `CMP_JUMP_*`, `FOREACH_BEGIN`, `FOREACH_NEXT`, `ITER_BEGIN` and `ITER_NEXT` continue and seed their target at `d`; `JUMP` and `LOOP` seed their target at `d` and terminate; `JUMP_TABLE` seeds all targets at `d−1` and terminates; `TRY_BEGIN` (at depth 0) seeds its catch target at 1 (the caught exception) and its finally target at 0; `TRY_END` and `CATCH_END` terminate (they go to the finally); a `break` / `continue` `TRY_LEAVE` seeds its target at 0 and terminates; `TAIL_CALL` and `TAIL_THIS_INVOKE` seed the next instruction at 1 (a callee that cannot reuse the frame returns onto the `RETURN` left standing) and terminate; `TAIL_SELF`, `RETURN*` and `THROW` terminate.

Appendix A. Opcode map

Dense numbering for the current opcode set. `OP_LAST` = 187 (not a real instruction). Any opcode ≥ 187 MUST be rejected.

#	Hex	Mnemonic	#	Hex	Mnemonic
0	0x00	`OP_NOP`	94	0x5E	`OP_TAIL_SELF`
1	0x01	`OP_CONSTANT`	95	0x5F	`OP_TAIL_CALL`
2	0x02	`OP_NIL`	96	0x60	`OP_RETURN`
3	0x03	`OP_TRUE`	97	0x61	`OP_RETURN_NONE`
4	0x04	`OP_FALSE`	98	0x62	`OP_RETURN_NIL`
5	0x05	`OP_IMM8`	99	0x63	`OP_RETURN_L`
6	0x06	`OP_IMM16`	100	0x64	`OP_FN`
7	0x07	`OP_IMM32`	101	0x65	`OP_FN_CAPTURE`
8	0x08	`OP_IMM_CHAR`	102	0x66	`OP_YIELD`
9	0x09	`OP_POP`	103	0x67	`OP_AWAIT`
10	0x0A	`OP_DUP`	104	0x68	`OP_THIS_INVOKE`
11	0x0B	`OP_NEGATE`	105	0x69	`OP_BIND_THIS`
12	0x0C	`OP_NOT`	106	0x6A	`OP_PUSH_BUILTIN`
13	0x0D	`OP_BITWISE_NOT`	107	0x6B	`OP_CALL0_BUILTIN`
14	0x0E	`OP_ADD`	108	0x6C	`OP_CALL1_BUILTIN`
15	0x0F	`OP_SUBTRACT`	109	0x6D	`OP_CALL2_BUILTIN`
16	0x10	`OP_MULTIPLY`	110	0x6E	`OP_CALL3_BUILTIN`
17	0x11	`OP_DIVIDE`	111	0x6F	`OP_CALL4_BUILTIN`
18	0x12	`OP_MODULO`	112	0x70	`OP_CALL5_BUILTIN`
19	0x13	`OP_POWER`	113	0x71	`OP_TRY_BEGIN`
20	0x14	`OP_BITWISE_AND`	114	0x72	`OP_TRY_END`
21	0x15	`OP_BITWISE_OR`	115	0x73	`OP_CATCH_BEGIN`
22	0x16	`OP_XOR`	116	0x74	`OP_CATCH_END`
23	0x17	`OP_SHL`	117	0x75	`OP_FINALLY_BEGIN`
24	0x18	`OP_SHR`	118	0x76	`OP_FINALLY_END`
25	0x19	`OP_USHR`	119	0x77	`OP_TRY_LEAVE`
26	0x1A	`OP_IS_NIL`	120	0x78	`OP_THROW`

#	Hex	Mnemonic	#	Hex	Mnemonic
27	0x1B	OP_IS_NOT_NIL	121	0x79	OP_FOREACH_BEGIN
28	0x1C	OP_EQUAL	122	0x7A	OP_FOREACH_NEXT
29	0x1D	OP_NOT_EQUAL	123	0x7B	OP_ITER_BEGIN
30	0x1E	OP_LESS	124	0x7C	OP_ITER_NEXT
31	0x1F	OP_LESS_EQUAL	125	0x7D	OP_LEN
32	0x20	OP_GREATER	126	0x7E	OP_TYPE
33	0x21	OP_GREATER_EQUAL	127	0x7F	OP_HAS_KEY
34	0x22	OP_APPROX_EQ	128	0x80	OP_TO_STRING
35	0x23	OP_CMP_IS	129	0x81	OP_TO_INT
36	0x24	OP_JUMP_IF_FALSE	130	0x82	OP_TO_FLOAT
37	0x25	OP_JUMP_IF_TRUE	131	0x83	OP_TO_CHAR
38	0x26	OP_JUMP	132	0x84	OP_TO_I8
39	0x27	OP_LOOP	133	0x85	OP_TO_U8
40	0x28	OP_JUMP_TABLE	134	0x86	OP_TO_I16
41	0x29	OP_CMP_JUMP_LL	135	0x87	OP_TO_U16
42	0x2A	OP_CMP_JUMP_LC	136	0x88	OP_TO_I32
43	0x2B	OP_DEFINE_GLOBAL	137	0x89	OP_TO_U32
44	0x2C	OP_GET_GLOBAL	138	0x8A	OP_EXPORT
45	0x2D	OP_SET_GLOBAL	139	0x8B	OP_NEW
46	0x2E	OP_GET_LOCAL	140	0x8C	OP_SUPER
47	0x2F	OP_SET_LOCAL	141	0x8D	OP_GUARD
48	0x30	OP_GET_LOCAL0	142	0x8E	OP_DBG_LINE
49	0x31	OP_GET_LOCAL1	143	0x8F	OP_DBG_FILE_NAME
50	0x32	OP_GET_LOCAL2	144	0x90	OP_DBG_BREAK
51	0x33	OP_GET_LOCAL3	145	0x91	OP_GET_BLK_LL
52	0x34	OP_SET_LOCAL0	146	0x92	OP_GET_BLK_LI
53	0x35	OP_SET_LOCAL1	147	0x93	OP_SET_BLK_LL
54	0x36	OP_SET_LOCAL2	148	0x94	OP_SET_BLK_LLL
55	0x37	OP_SET_LOCAL3	149	0x95	OP_CONCAT
56	0x38	OP_INC_LOCAL	150	0x96	OP_ARITH_FMA_LLL
57	0x39	OP_DEC_LOCAL	151	0x97	OP_INVOKE_INSTANCE
58	0x3A	OP_ARITH_LC	152	0x98	OP_CMP_JUMP_LC32
59	0x3B	OP_ARITH_LL	153	0x99	OP_GET_LOCAL_PROP
60	0x3C	OP_ARITH_L	154	0x9A	OP_GET_THIS_SLOT
61	0x3D	OP_ARITH_IMM8	155	0x9B	OP_SET_THIS_SLOT
62	0x3E	OP_ARITH_L_IMM8	156	0x9C	OP_GET_THIS_CONST
63	0x3F	OP_ARITH_LL_PUSH	157	0x9D	OP_THIS_ARITH_SLOT_L
64	0x40	OP_GET_PROPERTY	158	0x9E	OP_THIS_ARITH_SLOT_C
65	0x41	OP_SET_PROPERTY	159	0x9F	OP_THIS_INVOKE_SLOT
66	0x42	OP_GET_INDEX	160	0xA0	OP_SUPER_INVOKE
67	0x43	OP_SET_INDEX	161	0xA1	OP_CALL_GLOBAL
68	0x44	OP_GET_ARR_LC	162	0xA2	OP_CONCAT_N
69	0x45	OP_GET_ARR_LL	163	0xA3	OP_NIL_LOCAL
70	0x46	OP_GET_ARR_LI	164	0xA4	OP_CALL_WITH_THIS
71	0x47	OP_GET_INDEX_LOCAL	165	0xA5	OP_OBJ_ARITH_L
72	0x48	OP_SET_ARR_LC	166	0xA6	OP_PUSH_HOST
73	0x49	OP_SET_ARR_LL	167	0xA7	OP_CALL0_HOST
74	0x4A	OP_SET_ARR_LLC	168	0xA8	OP_CALL1_HOST
75	0x4B	OP_SET_ARR_LLL	169	0xA9	OP_CALL2_HOST
76	0x4C	OP_SET_INDEX_LOCAL	170	0xAA	OP_CALL3_HOST
77	0x4D	OP_ARITH_ELC	171	0xAB	OP_CALL4_HOST
78	0x4E	OP_ARITH_ELL	172	0xAC	OP_CALL5_HOST

#	Hex	Mnemonic	#	Hex	Mnemonic
79	0x4F	OP_ARITH_EIC	173	0xAD	OP_INVOKE_MEMBER
80	0x50	OP_ARITH_EIL	174	0xAE	OP_INVOKE_GLOBAL_CACHED
81	0x51	OP_SET_OBJ_L	175	0xAF	OP_TAIL_THIS_INVOKE
82	0x52	OP_SET_OBJ_LL	176	0xB0	OP_TAIL_THIS_INVOKE_SLOT
83	0x53	OP_GET_THIS_PROP	177	0xB1	OP_SUPER_INVOKE_SLOT
84	0x54	OP_SET_THIS_PROP	178	0xB2	OP_ARITH_IMM16
85	0x55	OP_THIS_ARITH_L	179	0xB3	OP_CHECK_TYPE
86	0x56	OP_THIS_ARITH_C	180	0xB4	OP_OBJ_ARITH_S
87	0x57	OP_LOCAL_ARR_LC	181	0xB5	OP_CMP_C
88	0x58	OP_LOCAL_ARR_LL	182	0xB6	OP_GET_MAT_LLL
89	0x59	OP_BUILD_OBJECT	183	0xB7	OP_SET_MAT_LLL
90	0x5A	OP_BUILD_ARRAY	184	0xB8	OP_ARITH_MLL
91	0x5B	OP_MAKE_CONST	185	0xB9	OP_ARITH_MLC
92	0x5C	OP_CALL	186	0xBA	OP_CONSTANT_REF
93	0x5D	OP_CALL_SELF			

Appendix B. Machine limits and named constants

This appendix is the normative value table for **every** named constant used in this document — the specification is self-contained and requires no access to the VM's source code. A conforming implementation MUST enforce the same caps where they are load-time validation rules, and SHOULD match them at run time for portability of programs near the limits.

B.1 Format constants

Constant	Value	Meaning
FLS_HEADER_SIZE	192	`.flx` file-header size in bytes (§5.2)
CHUNK_MAGIC	0xA5	first byte of every serialized chunk (§5.5)
FLX_POOL_REF	0xFFFFFF01	type-tag word marking a string-pool reference (§5.4)
FLX_POOL_MAX	65535	maximum string-pool entries per code region (§5.4)
BUNDLE_ENTRY_SIZE	152	bundle TOC entry size in bytes (§5.7)
MAX_BUNDLE_DEPS	1024	maximum dependencies in one bundle TOC (§5.7)
FLS_SIG_CONTEXT	"flaris.flx.sig.v1"	signature domain-separation string (§5.3)
XXHASH64_SEED	0	seed for all identifier hashing (§4.6)
XXHASH64_EMPTY	0xEF46DB3751D8E999	xxHash64 of the empty string (§4.6)
EXIT_CODE_ERR_FILE	4	process exit code for every load-time rejection (§8)
MAX_VALIDATE_NESTING	256	validator recursion cap over nested function chunks (§8.3)

B.2 Machine limits

Constant	Value	Applies to
MAX_FIBERS	1024	live fibers (MIN_FIBERS 2, default 256)
MAX_STACK	65535	operand-stack slots per fiber (MIN_STACK 64, default 1024)
MAX_CALL_FRAMES	1024	call depth per fiber (MIN_CALL_FRAMES 8, default 64)
MAX_LOCALS	128	local slots per frame
MAX_CALL_ARGUMENTS	16	arguments per call (+2 internal slots for bound methods)
MAX_BUILTIN_ARGUMENTS	6	declared builtin parameters
MAX_FL_EXCEPTION_LEVELS	24	nested try contexts per fiber
MAX_PROTO_NEST	8	class-inheritance depth
MAX_CHUNK_DEPTH	64	nested-chunk depth in a `.flx`

Constant	Value	Applies to
`MAX_CODE_SIZE`	10 MiB	bytecode bytes per chunk
`MAX_CONST_COUNT`	16384	constants per chunk
`MAX_ITEMS`	16 777 216 (2^24)	array elements / object properties
`MAX_STRING_SIZE`	256 MiB	string bytes
`MAX_MEM_BLOCK_ALLOC`	2 GiB	one memory block
`MAX_TOTAL_BLOCK_ALLOCATION`	64 GiB	total block memory
`MAX_BLOCK_ELEM_SIZE`	255	block element size (stored in one byte)
`MAX_EVAL_COMPILE_SRC_LEN`	10 KiB	`Eval` / `Compile` source length
`MAX_CONCAT_FUSE`	16	operands of one `OP_CONCAT_N` (§7.21)
`MAX_CLONE_DEPTH`	512	deep-clone recursion (container `+`, `Clone`) before `NestingError`
`MAX_COMPARE_DEPTH`	1024	structural-comparison recursion before `NestingError`
`MAX_PRINT_DEPTH`	64	console value printing descends this deep, then elides
`FIBER_QUANTUM`	10000	scheduling checkpoints per time slice
`MAILBOX_CAP`	64	per-fiber mailbox depth
`MAX_EVENTS`	256	event objects
`SSO_MAX_CHARS`	7	small-string optimization threshold (implementation note, §4.4 — not observable)

Appendix C. Worked example

A minimal program (`spec_check.fls`):

```
fn Add(a, b)  { return a + b; }
fn Main()     { let total = 0;
                for (let i = 0; i < 10; i++) { total += Add(i, 2); }
                Console.WriteLine(total); }
```

compiled with `flarisvm --compile spec_check.fls spec_check.flx` (unsigned) produces a 627-byte file. Annotated prefix (all values little-endian):

```
offs  bytes                     field
0000  46 4C 53 32               magic "FLS2"
0004  00 00 00 01               library version 0x01000000  (1.0.0.0)
0008  08 00 00 01               bytecode version 0x01000008 (1.0.0.8)
000C  73 70 65 63 5F 63 68 65   name "spec_check\0" (basename, no extension)
      63 6B 00 …
002C  00 00 00 00               flags = 0 (not a bundle)
0030  DC BE 50 90 22 3A 78 C6   entry = 0xC6783A229050BEDC = xxhash64("Main")
0038  C0 00 00 00  C0 00 00 00  mainSectionStart = stringPoolStart = 192
0040  FE 00 00 00  FE 00 00 00  stringPoolEnd = codeStart = 254
0048  73 02 00 00  73 02 00 00  codeEnd = mainSectionEnd = 627 (= file size)
0052  00                        sigAlg = 0 (unsigned)
0054  00 × 32                  pubkey (zero)
0074  00 × 64                  signature (zero)
00B4  00 × 12                  padding
                               ── string pool (§5.4) ──
```

```
00C0  03 00                     pool count = 3
00C2  00                        [0] isStatic
00C3  57 77 68 5F D7 66 D3 1A   [0] hash = 0x1AD366D75F687757 = xxhash64("Add")
00CB  03 00 00 00  41 64 64     [0] len 3, "Add"
00D2  00                        [1] isStatic
00D3  62 3C 10 EF B6 40 DB 65   [1] hash
00DB  0E 00 00 00  73 70 65 …   [1] len 14, "spec_check.fls"
00EA  00                        [2] isStatic
00EB  DC BE 50 90 22 3A 78 C6   [2] hash = xxhash64("Main")
00F3  04 00 00 00  4D 61 69 6E  [2] len 4, "Main"
                                ── top-level chunk (§5.5) ──
00FE  A5                        chunk magic
00FF  08 00 00 01               chunk version 0x01000008
0103  04 00                     consts = 4
0105  13 00 00 00               codelen = 19
0109  …                         flags, then constants / code / debug per §5.5
```

The three pooled strings are exactly the strings that occur more than once in the value tree ("Add" and "Main" appear as both function name and global key; the source path appears in both functions' debug info) — single-use strings stay inline (§5.4).

Appendix D. Source-to-opcode examples

One row per opcode: a minimal source construct that emits it, in the same grouping and order as §7. "Local" in an example means a genuine local variable (parameter or `let`/`var`) — a function call result or property read never qualifies, which is why several rows use `foo()` specifically to force the generic/pushed form rather than a fused local form. Unless a row's comment says otherwise, every example was confirmed by tracing the compiled chunk, not inferred from the prose alone.

D.1 Stack and constants (§7.1)

Code	Opcode(s)	Comment
(none — peephole filler)	`OP_NOP`	Not compiler-emitted from source. The bytecode-level peephole pass pads reclaimed bytes with `OP_NOP` when it fuses a `this.field[index]` read or a call through a global into one opcode, so later jump offsets don't shift. A tail call keeps the `RETURN` after it: a callee that cannot reuse the frame returns onto it.
`let s = "hello";`	`OP_CONSTANT`	String constants are copied out of the pool on every push — strings are mutable at runtime, the pool object must not be aliased.
`let x = nil;`	`OP_NIL`	
`let x = true;`	`OP_TRUE`	
`let x = false;`	`OP_FALSE`	
`let x = 5;`	`OP_IMM8`	Every int literal in i8 range, `0`, `1` and `-1` included.
`let x = 1000;`	`OP_IMM16`	
`let x = 100000;`	`OP_IMM32`	
`let c = 'A';`	`OP_IMM_CHAR`	

Code	Opcode(s)	Comment
`foo();` (statement)	`OP_POP`	Every expression statement whose value nothing uses discards it.
`switch (x) { case 1: ... }`	`OP_DUP`	Duplicates the scrutinee before each case's equality test, since the original must survive for the next comparison.

D.2 Arithmetic and logic (§7.2)

Code	Opcode(s)	Comment
`let y = -foo();`	`OP_NEGATE`	
`let y = !foo();`	`OP_NOT`	
`let y = ~foo();`	`OP_BITWISE_NOT`	
`let r = foo() + bar();`	`OP_ADD`	Neither operand is a local, so none of §7.6's fused forms apply — the fully generic lane-dispatching add.
`let r = foo() - bar();`	`OP_SUBTRACT`	
`let r = foo() * bar();`	`OP_MULTIPLY`	
`let r = foo() / bar();`	`OP_DIVIDE`	
`let r = foo() % bar();`	`OP_MODULO`	
`let r = foo() ^^ bar();`	`OP_POWER`	
`let r = foo() & bar();`	`OP_BITWISE_AND`	
`let r = foo() | bar();`	`OP_BITWISE_OR`	
`let r = foo() ^ bar();`	`OP_XOR`	
`let r = foo() << bar();`	`OP_SHL`	
`let r = foo() >> bar();`	`OP_SHR`	
`let r = foo() >>> bar();`	`OP_USHR`	

D.3 Comparison (§7.3)

Code	Opcode(s)	Comment
`if (x == nil) { ... }`	`OP_IS_NIL`	The compiler special-cases a `nil` literal on either side of `==` / `!=` instead of a generic `OP_EQUAL`.
`if (x != nil) { ... }`	`OP_IS_NOT_NIL`	
`if (foo() == bar()) { ... }`	`OP_EQUAL`	Neither side is a nil literal, so this is the generic compare.
`if (foo() != bar()) { ... }`	`OP_NOT_EQUAL`	
`if (foo() < bar()) { ... }`	`OP_LESS`	Both operands non-local — §7.4's `CMP_JUMP_LL` needs bare locals, so this falls to the generic push-then-compare.

Code	Opcode(s)	Comment
`if (foo() <= bar()) { ... }`	`OP_LESS_EQUAL`	
`if (foo() > bar()) { ... }`	`OP_GREATER`	
`if (foo() >= bar()) { ... }`	`OP_GREATER_EQUAL`	
`if (a ~= b) { ... }`	`OP_APPROX_EQ`	
`if (x is MyClass) { ... }`	`OP_CMP_IS`	
`if (cmd == "add") { ... }`	`OP_CMP_C`	A constant right operand: the peephole pass fuses `CONSTANT` with the comparison, so the literal is compared where it sits instead of being copied onto the stack first. Covers `==`, `!=`, `<`, `<=`, `>`, `>=`.

D.4 Control flow (§7.4)

Code	Opcode(s)	Comment
`if (cond) { ... }` `a || b`	`OP_JUMP_IF_FALSE` `OP_JUMP_IF_TRUE`	Also every short-circuit `&&`'s first operand.
`if (c) { ... } else { ... }`	`OP_JUMP`	Skips the else-branch after the then-branch runs.
`while (cond) { ... }`	`OP_LOOP`	The mandatory backward branch/quantum checkpoint at every loop back-edge.
`switch (x) { case 1: ...; case 2: ...; }`	`OP_JUMP_TABLE`	Only when every case label is an int literal and the value range is small/dense enough — a string switch, or a sparse int switch, falls to the `OP_DUP` + `OP_EQUAL` comparison chain instead.
`while (i < n) { ... }`	`OP_CMP_JUMP_LL`	`i`, `n` both locals — fuses `GET_LOCAL×2 + CMP + JUMP_IF_FALSE` into one zero-stack op. Applies to `if` / `while` / `for` conditions alike, not just loops.
`while (i < 10) { ... }`	`OP_CMP_JUMP_LC`	`i` local, immediate in i8 range.
`while (i < 100000) { ... }`	`OP_CMP_JUMP_LC32`	Same shape, immediate too large for i8.
`if (x >= lo && x <= hi) { ... }`	`OP_CMP_JUMP_LL` ×2	Each side of an `&&` / `||`

D.5 Globals and locals (§7.5)

Code	Opcode(s)	Comment
`global x = 5;`	`OP_DEFINE_GLOBAL`	
`Console.WriteLine(globalVar);`	`OP_GET_GLOBAL`	Reads a global or a closure-captured name (resolved through the environment chain either way).
`globalVar = 5;`	`OP_SET_GLOBAL`	Rebinds an existing global/captured binding.
`return x;` (`x` in slot ≥ 4)	`OP_GET_LOCAL`	
`x = foo();` (`x` a local, rhs not fusable)	`OP_SET_LOCAL`	
`return x;` (`x` in slot 0-3, e.g. the first parameter)	`OP_GET_LOCAL0` … `OP_GET_LOCAL3`	Dedicated 1-byte forms for the four most common slots.
`x = foo();` (`x` in slot 0-3)	`OP_SET_LOCAL0` … `OP_SET_LOCAL3`	
`x++;`	`OP_INC_LOCAL`	Also `x += 1;` (canonicalized to the same form before codegen).

Code	Opcode(s)	Comment
`x--;`	`OP_DEC_LOCAL`	
`while (...) { let acc = 0; ... }`	`OP_NIL_LOCAL`	Clears a loop-body local at the back-edge before the next iteration re-declares it, so iteration N doesn't keep iteration N-1's value alive through the slot.

D.6 Local compound arithmetic (§7.6)

Code	Opcode(s)	Comment
`x += 5;`	`OP_ARITH_LC`	
`x += y;` (`y` local)	`OP_ARITH_LL`	Compound-assign form — `x` is both source and destination, distinct from `ARITH_LL_PUSH` below.
`x += foo();`	`OP_ARITH_L`	rhs not local/immediate, so it's computed on the stack first.
`let r = foo() + 5;`	`OP_ARITH_IMM8`	`foo()` isn't a bare local, so the `L_IMM8` fusion below doesn't apply.
`let r = x + 5;` (`x` local)	`OP_ARITH_L_IMM8`	Fresh push, not a compound-assign — the `GET_LOCAL`-free form of `ARITH_IMM8`.
`let r = x + y;` (`x`, `y` both local)	`OP_ARITH_LL_PUSH`	Closes the gap `ARITH_L_IMM8` leaves for two-local operands; ~15-20% faster than the `push+push+ADD` it replaces on a tight loop.
`let r = a*b+c;` / `return a*b+c;` / `f(a*b+c)` (`a`, `b`, `c` all local, all proven int)	`OP_ARITH_FMA_LLL`	Never emitted when any operand may be a float: `a*b+c` on floats stays a separate multiply and add. Also reached from `arr[a*b+c]` / `buf[a*b+c]` (§D.8/§D.20) via a compiler-allocated temp holding the FMA result, and from any plain pushed-value position — a return expression, a call argument, a nested sub-expression — not just an assignment target or an index. Measured ~9% faster than the unfused sequence in the pure-interpreter case (JIT disabled), rising to ~14% once the temp is correctly recognized as never needing a per-iteration clear inside a loop body (the same recognition also improved the array/block indexed-access temps in §D.8/§D.20 by a further ~2-9% on top of their own already-measured wins). A JIT-compiled, fully-typed hot function bypasses this fusion entirely, since the JIT compiles from source through its own independent code path rather than executing this bytecode.

D.7 Property and index access, generic (§7.7)

Code	Opcode(s)	Comment
`foo().field`	`OP_GET_PROPERTY`	Receiver not a local.
`let n = obj.name;` (`obj` local)	`OP_GET_LOCAL_PROP`	Fused `GET_LOCAL + GET_PROPERTY` for any local receiver, `this` included when it isn't otherwise specialized.
`foo().field = v;`	`OP_SET_PROPERTY`	
`getContainer()[k]`	`OP_GET_INDEX`	Container not from a local — the fully generic form.
`getContainer()[k] = v;`	`OP_SET_INDEX`	

D.8 Array fast paths (§7.8)

Code	Opcode(s)	Comment
`a[5]`	`OP_GET_ARR_LC`	
`a[i]`	`OP_GET_ARR_LL`	
`a[f(x)]`	`OP_GET_ARR_LI`	Computed, non-FMA-shaped index — reads are already optimal here (index consumed straight off the stack), so the temp-index trick used for writes (below) deliberately does not apply to reads.
`obj[key]` (`obj` local, type not proven	`OP_GET_INDEX_LOCAL`	Also `a[k]` when `k` isn't compiler-proven int.

Code	Opcode(s)	Comment
array)		
`a[5] = v;`	`OP_SET_ARR_LC`	
`a[i] = v;`	`OP_SET_ARR_LL`	
`a[5] = j;` (`j` local)	`OP_SET_ARR_LLC`	Zero-stack.
`a[i] = j;` (`j` local)	`OP_SET_ARR_LLL`	Zero-stack.
`obj[key] = v;` (generic fallback)	`OP_SET_INDEX_LOCAL`	A computed index (`a[f(x)] = v`) on a proven-array receiver is instead routed through a temp into `SET_ARR_LL` / `LLL` — measured -14 to -15% on a hot loop versus falling here.
`a[i] += 1;`	`OP_ARITH_ELC`	
`a[i] += j;`	`OP_ARITH_ELL`	
`a[5] += 1;`	`OP_ARITH_EIC`	Closes the gap left by a 9-op generic fallback sequence; measured -38%.
`a[5] += j;`	`OP_ARITH_EIL`	Same fusion, local right-hand side.
`a[f(x)] += 1;`	`OP_ARITH_ELC` / `ELL` (via temp)	The computed index is evaluated into a compiler temp, then addressed as if it were a local — measured -14 to -15% versus the 9-op generic sequence the plain-write case above also used to fall to.
`let x = a[5];`	`OP_LOCAL_ARR_LC`	
`let x = a[i];`	`OP_LOCAL_ARR_LL`	
`let x = a[f(x)];`	`OP_GET_ARR_LI` + `OP_SET_LOCAL`	**No dedicated `LOCAL_ARR_LI`** — costs a push+pop that a fused form would save. Smallest of the catalogued gaps (one dispatch, not a full generic-path cliff); not yet implemented.

D.9 Object property fast paths (§7.9)

Code	Opcode(s)	Comment
`obj.field = foo();` (`obj` local)	`OP_SET_OBJ_L`	
`obj.field = j;` (`j` local)	`OP_SET_OBJ_LL`	Zero-stack.
`obj.field += j;`	`OP_OBJ_ARITH_L`	Exists specifically so a string field's `+=` can append in place — the generic `DUP` / `GET_PROPERTY` /op/ `SET_PROPERTY` sequence would give the field a second reference and make the append (and the surrounding loop) quadratic.
`obj.field -= dx * m;`	`OP_OBJ_ARITH_S`	The same with an expression on the right, when it has no effects: the expression, then one read-modify-write of the field.

D.10 `this` access (§7.10)

Code	Opcode(s)	Comment
`this.field` (first execution)	`OP_GET_THIS_PROP`	Self-patches after the first call — see the next two rows.
`this.field = v;` (first execution)	`OP_SET_THIS_PROP`	
`this.field` (after self-patch, field member)	`OP_GET_THIS_SLOT`	**Never compiler-emitted** — the VM rewrites `GET_THIS_PROP` in place once it knows the member is a field. Same source, different opcode depending on how many times it's run.
`this.field = v;` (after self-patch)	`OP_SET_THIS_SLOT`	Never compiler-emitted; VM-patched from `SET_THIS_PROP`.
`this.MAX_SIZE` (after self-patch, const member)	`OP_GET_THIS_CONST`	Never compiler-emitted; `GET_THIS_PROP` patches here instead of `GET_THIS_SLOT` when the member turns out to

Code	Opcode(s)	Comment
		be a class const, not a field.
`this.count += j;` (first execution)	`OP_THIS_ARITH_L`	
`this.count += 1;` (first execution)	`OP_THIS_ARITH_C`	
`this.count += j;` (after self-patch)	`OP_THIS_ARITH_SLOT_L`	Never compiler-emitted.
`this.count += 1;` (after self-patch)	`OP_THIS_ARITH_SLOT_C`	Never compiler-emitted.
`this.items[i]`	`OP_GET_THIS_PROP`, `OP_GET_LOCAL`, `OP_GET_INDEX`	The field read patches itself into `OP_GET_THIS_SLOT`; the implicit bare `items[i]` compiles the same.
`this.items[i] = v;`	`OP_GET_THIS_PROP`, `OP_GET_LOCAL`, *v*, `OP_SET_INDEX`	Left to right: the field and the index are read before `v` runs. The implicit bare `items[i] = v;` compiles the same.
`fn() { return this.x; }` (closure literal inside a method)	`OP_BIND_THIS`	
`new C() { this.x = 1; }`	`OP_CALL_WITH_THIS`	The init-block form; compiled as an anonymous thiscall function.

D.11 Construction (§7.11)

Code	Opcode(s)	Comment
`{a: 1, b: 2}`	`OP_BUILD_OBJECT`	
`[1, 2, 3]`	`OP_BUILD_ARRAY`	Elements pushed in source order; also the flat-int/flat-float path when the element type is provably `int` / `float` throughout.
`const arr = [1, 2, 3];`	`OP_MAKE_CONST`	Reference-type `const` only — a scalar `const` needs no runtime marker.
`new MyClass(args)`	`OP_NEW`	
`super(args);`	`OP_SUPER`	Pushes the base-class constructor bound to `this`; the actual call is a following `OP_CALL`.
`x!`	`OP_GUARD`	Non-null assertion.

D.12 Calls and returns (§7.12)

Code	Opcode(s)	Comment
`f(a, b)`	`OP_CALL`	The argument types are checked at run time.
`fib(n - 1);` (inside `fn fib`, non-tail)	`OP_CALL_SELF`	Self-recursive call, no callee lookup, no type check. Falls back to `OP_CALL` if the argument types aren't provably safe.
`return fib(n - 1);` (self, tail position)	`OP_TAIL_SELF`	Rebinds the current frame's locals and restarts at ip 0 — constant stack and frame depth. Arity-checked before the rebind.
`return other(x);` (tail call, different function)	`OP_TAIL_CALL`	True tail-call elimination for a plain-function callee.
`return foo();`	`OP_RETURN`	Generic — `foo()`'s result isn't a bare local.
(loader-synthesized, empty top-level chunk)	`OP_RETURN_NONE`	Not compiler-emitted from source at all; written by the `.flx` writer as a top-level chunk's terminator.
`fn f() { }` (falls off the end, no explicit `return`)	`OP_RETURN_NIL`	The implicit epilogue every function body gets.
`return x;` (`x` a bare local)	`OP_RETURN_L`	Fused `GET_LOCAL + RETURN`.
`fn foo() { }` (module scope)	`OP_FN`	A function literal *inside* a function body gets `OP_FN_CAPTURE` instead, even with nothing to capture — see below.

Code	Opcode(s)	Comment
`fn() { return x; }` (closure literal, any scope other than module-level)	`OP_FN_CAPTURE`	
`yield v;`	`OP_YIELD`	
`await fiberExpr;`	`OP_AWAIT`	

D.13 Method invocation (§7.13)

Code	Opcode(s)	Comment
`obj.method(args)` (receiver's exact kind not proven)	`OP_INVOKE_MEMBER`	Self-classifying inline cache — see §7.13. A plain (non-class, non-instance) receiver stays on the generic resolution, uncached.
`this.method(args);` (first execution)	`OP_THIS_INVOKE`	
`this.method(args);` (after self-patch)	`OP_THIS_INVOKE_SLOT`	Never compiler-emitted; re-reads the vtable slot every call so overrides/hot-patches stay correct without re-patching.
`return this.method(args);` (tail position, first execution)	`OP_TAIL_THIS_INVOKE`	Reuses the current frame for a plain, non-async callee — see §7.13.
`return this.method(args);` (tail position, after self-patch)	`OP_TAIL_THIS_INVOKE_SLOT`	Never compiler-emitted; same frame-reuse eligibility as `OP_TAIL_THIS_INVOKE`.
`obj.method(args)` (`obj` proven a class instance)	`OP_INVOKE_INSTANCE`	Carries an inline cache keyed by the receiver's class layout.
`Math.Max(a, b);` (dotted call, global/module receiver)	`OP_INVOKE_GLOBAL_CACHED`	Self-classifying method resolution on a class receiver — see §7.13. The global lookup itself is unchanged; a module receiver stays fully uncached.
`super.method(args);` (first execution)	`OP_SUPER_INVOKE`	Resolves on the base of the *defining* class, not the receiver's dynamic class.
`super.method(args);` (after self-patch)	`OP_SUPER_INVOKE_SLOT`	Never compiler-emitted; base is fixed per call site so no classId guard is needed.
`someGlobalFn(args);`	`OP_CALL_GLOBAL`	Fused `GET_GLOBAL + CALL`; emitted after tail-call optimization so a tail-positioned global call still gets TCO.

D.14 Builtin call shortcuts (§7.14)

Code	Opcode(s)	Comment
`let f = Console.WriteLine;`	`OP_PUSH_BUILTIN`	A builtin referenced as a value rather than called directly.
`Time.Millis();`	`OP_CALL0_BUILTIN`	
`Console.WriteLine("hi");`	`OP_CALL1_BUILTIN`	
`Console.WriteLine("x=", x);`	`OP_CALL2_BUILTIN`	
(a 3-argument builtin call)	`OP_CALL3_BUILTIN`	
(a 4-argument builtin call)	`OP_CALL4_BUILTIN`	
(a 5-argument builtin call)	`OP_CALL5_BUILTIN`	`CALL0..5_BUILTIN` are emitted as `OP_CALL0_BUILTIN + argc`, not by name — one codegen site covers all six.

D.14a Host call shortcuts (§7.14a)

Only reachable in an embedding that registers its own native modules (host options or `FlarisRegisterModule`) — not exercised by any `.fls` in this repo.

Code	Opcode(s)	Comment
`let f = Device.Read;` (host-registered module)	`OP_PUSH_HOST`	
`Device.Poll();`	`OP_CALL0_HOST`	
`Device.Read(fd);`	`OP_CALL1_HOST`	
(2/3/4/5-argument host calls)	`OP_CALL2_HOST` … `OP_CALL5_HOST`	Same `OP_CALL0_HOST + argc` pattern as the builtin shortcuts.

D.15 Exception handling (§7.15)

Code	Opcode(s)	Comment
`try { ... } catch (e) { ... }`	`OP_TRY_BEGIN`	
(try body completes normally)	`OP_TRY_END`	
(catch body entry)	`OP_CATCH_BEGIN`	
(catch body completes normally)	`OP_CATCH_END`	
`finally { ... }` (entry)	`OP_FINALLY_BEGIN`	
`finally { ... }` (exit)	`OP_FINALLY_END`	
`return x;` inside a `try` that has an enclosing `finally`	`OP_TRY_LEAVE`	Also `break` / `continue` out of a try/catch guarded by a `finally`.
`throw new Exception("msg");`	`OP_THROW`	

D.16 Iteration (§7.16)

Code	Opcode(s)	Comment
`foreach (item in arr) { ... }` (loop entry, once)	`OP_FOREACH_BEGIN`	Also works over strings (by byte), objects/instances (by map entry).
`foreach (item in arr) { ... }` (back-edge, per iteration)	`OP_FOREACH_NEXT`	
`iter (i from 0 to 11) { ... }` (loop entry, once)	`OP_ITER_BEGIN`	
`iter (i from 0 to 11) { ... }` (back-edge, per iteration)	`OP_ITER_NEXT`	

D.17 Type operations (§7.17)

Code	Opcode(s)	Comment
`len(x)`	`OP_LEN`	Bare global builtin call, distinct from `.Length` (a separate member builtin on string/array).
`type(x)`	`OP_TYPE`	Same family of bare global builtins; `x`'s `ObjectType` bit, compare against `Type.*` constants.
`if (k in c) { ... }`	`OP_HAS_KEY`	
`str(x)` / `(string)x`	`OP_TO_STRING`	Both the bare-call and cast forms lower here.
`int(x)` / `(int)x`	`OP_TO_INT`	Both forms; also the casts `(u64)x` and `(i64)x`, which do not narrow.
`float(x)` / `(float)x`	`OP_TO_FLOAT`	
`char(x)` / `(char)x`	`OP_TO_CHAR`	
`i8(x)` / `(i8)x`	`OP_TO_I8`	Both forms, as for the five rows below.
`u8(x)` / `(u8)x`	`OP_TO_U8`	
`i16(x)` / `(i16)x`	`OP_TO_I16`	
`u16(x)` / `(u16)x`	`OP_TO_U16`	
`i32(x)` / `(i32)x`	`OP_TO_I32`	
`u32(x)` / `(u32)x`	`OP_TO_U32`	

Code	Opcode(s)	Comment
`let n: int = f(x)` / `n = o.k` /… `return f(x)` in `fn g(): int`	`OP_CHECK_TYPE`	When the stored value's type is not known at compile time, or a known int or char must be converted; a statically known misfit is a compile error instead.

D.18 Modules (§7.18)

Code	Opcode(s)	Comment
`export const x = 5;`	`OP_EXPORT`	Also `export { name };` for a binding declared earlier in the module.

D.19 Debug (§7.19)

Code	Opcode(s)	Comment
(each change of source line, with `-g` only)	`OP_DBG_LINE`	Two statements on one line share one marker.
(the top-level chunk, once)	`OP_DBG_FILE_NAME`	Names the whole unit; nested chunks inherit it. Emitted with or without `-g`.
`breakpoint(7);`	`OP_DBG_BREAK`	Emitted with or without `-g`.

D.20 Block (raw memory) fast paths (§7.20)

Code	Opcode(s)	Comment
`buf[5]`	`OP_IMM8`, `OP_GET_BLK_LI`	A literal index goes on the stack, as a computed one does.
`buf[i]`	`OP_GET_BLK_LL`	
`buf[f(x)]`	`OP_GET_BLK_LI`	Previously fell to `OP_GET_INDEX_LOCAL`; measured -14% on a hot loop. Reads don't need the temp-index trick used for writes, same reasoning as the array case.
`buf[5] = v;`	`OP_IMM8`, `OP_SET_LOCAL`, `OP_SET_BLK_LL`	The literal index goes into a temp local, as a computed one does.
`buf[i] = v;`	`OP_SET_BLK_LL`	
`buf[5] = j;` (`j` local)	`OP_IMM8`, `OP_SET_LOCAL`, `OP_SET_BLK_LLL`	Zero-stack value; the literal index goes into a temp local.
`buf[i] = j;` (`j` local)	`OP_SET_BLK_LLL`	Zero-stack.
`let x = buf[5];`	`OP_IMM8`, `OP_GET_BLK_LI`, `OP_SET_LOCAL`	
`let x = buf[i];`	`OP_GET_BLK_LL`, `OP_SET_LOCAL`	
`buf[i] += 1;`	`OP_GET_BLK_LL`, `OP_ARITH_IMM8`, `OP_SET_BLK_LL`	A block has no compound-assign instruction: the element is read, the operator applied and the result stored, exactly as `buf[i] = buf[i] + 1`.
`buf[i] += j;`	`OP_GET_BLK_LL`, `OP_GET_LOCAL`, `OP_ADD`, `OP_SET_BLK_LL`	
`buf[5] += 1;`	`OP_IMM8`, `OP_SET_LOCAL`, `OP_GET_BLK_LL`, `OP_ARITH_IMM8`, `OP_SET_BLK_LL`	The literal index goes into a temp local, as for a store.
`buf[f(x)] += 1;`	*f(x)*, `OP_SET_LOCAL`, `OP_GET_BLK_LL`, `OP_ARITH_IMM8`, `OP_SET_BLK_LL`	A computed int index goes into a temp local the same way.

D.21 String concatenation (§7.21)

Code	Opcode(s)	Comment
`"a" + "b"`	`OP_CONCAT`	Both operands compiler-proven string/char; exactly two operands.
`"a" + "b" + "c"`	`OP_CONCAT_N`	Three or more proven-string/char leaves in one `+` chain — one allocation instead of a cascade of pairwise concats that would recopy every prefix.

Open items, not yet implemented: a dedicated `LOCAL_ARR_LI` / `LOCAL_BLK_LI` for `let x = a[f(x)]` (§D.8, §D.20). Separately: none of the fusion opcodes described in this appendix are recognized by the JIT compiler — it compiles from source through its own independent code path rather than executing this bytecode, so these interpreter-level fusions only matter for code that isn't JIT-compiled (untyped functions, or the JIT explicitly disabled).

Appendix D. Container format diagram

The `.flx` bytecode file format

Magic **FLS2** · little-endian throughout · Ed25519-signable · shared string pool · portable fixed-width integers

1 · File layout (byte offset increases →)

Ed25519 signature covers SHA-256 of the whole file (signature field zeroed)

Header	Bundle TOC	String pool	Code region
192 B, fixed	only if FLAG_BUNDLE	deduped strings	main chunk, with nested function chunks inline

@0 @192

main section = string pool + code · the unit a bundle copies per dependency (offsets @56–@76)

2 · Header — 192 bytes, fixed layout

off	sz	field	
@0	4	magic	"FLS2"
@4	4	version	format M.m.p.b (u32)
@8	4	compiler_version	u32
@12	32	name	output basename, NUL-term ‡
@44	4	flags	bit0 = FLAG_BUNDLE
@48	8	entry	entry-fn hash · 0 = library
@56	4	main_section_start	
@60	4	string_pool_start	
@64	4	string_pool_end	
@68	4	code_start	↑ absolute, validated on load

off	sz	field	
@72	4	code_end	
@76	4	main_section_end	
@80	2	(reserved)	
@82	1	sig_alg	0 = unsigned · 2 = Ed25519
@84	32	pubkey	Ed25519 public key
@116	64	signature	Ed25519 signature
@180	12	(padding → 192)	

Ed25519 signing block (@82–@179)

Signs the domain-separated message
`"flaris.flx.sig.v1" ‖ 0x00 ‖ SHA-256(file, sig=0)`
Trust anchors compiled in + ~/.flaris/trusted_keys

3 · Chunk — recursive (the code region is one chunk; each function constant nests another)

ChunkHeader	constants[consts]	code[codelen]	debug info	JIT IR
13 B	values (see 4)	bytecode · LE operands	if CHUNK_FLAG_DBG	if CHUNK_FLAG_HAS_JIT_IR

`ChunkHeader = magic u8 (0xA5) · version u32 · consts u16 · codelen u32 · flags u16`

flags: DBG = 0x0001 · HAS_JIT_IR = 0x0002. A `function` constant carries a whole nested Chunk — the loader caps nesting at depth 64 (MAX_CHUNK_DEPTH).
A single-byte `is_const` flag follows every value's type tag (see 4); the string pool (5) is read once, before the top chunk.

4 · Constant / value encoding

inline value
`[ type : u32 ] [ is_const : u8 ] [ payload ]`

pool reference → shared string pool (see 5)
`[ 0xFFFFFF01 : u32 ] [ slot : u16 ]`

payload by type

nil	— (no payload)	string	hash u64 · len u32 · bytes[len]
bool	u8	array	count u32 · flat u8 · elements…
char	u32	object	count u32 · (key,value) × count
int	i64	function	arity u8 · locals u8 · flags u32 · ret u32 · arg_types u32×16 · name (value) · nested Chunk
float	f64		

Strings appearing more than once in a file are stored once in the pool and emitted as pool references, shrinking shipped `.flx` ~15–30%.

5 · String pool — every repeated string stored once

Lives at `string_pool_start..string_pool_end` (header @60/@64), inside the main section, read once before the top chunk.

count : u16	entry 0	entry 1	entry … N-1
N entries	slot 0	slot 1	

pool entry (variable length)

`[ is_static : u8 ] [ hash : u64 (xxHash64) ] [ len : u32 ] [ bytes : len ]`

A constant references entry `i` via the pool-reference value `0xFFFFFF01, slot=i` (see 4). `is_static` is advisory (ignored on load); each reference takes its own ref on the shared Object.

6 · Bundle (FLAG_BUNDLE) — a self-contained, multi-module `.flx`

Built with `flarisvm --compile app.fls out.flx --bundle=Logger.flx;Router.flx`. FLAG_BUNDLE (header @44 bit0) is set; the VM pre-loads every embedded dependency before execution — no filesystem access for them. Cap: `MAX_BUNDLE_DEPS = 1024`.

File structure of a bundle (example: app + 2 deps)

Header	TOC	app · main section	Logger · main section	Router · main section
FLAG_BUNDLE	count=2 + 2 entries	pool + code	dep 0	dep 1
@0	@192	main_section_start	entry[0].offset	entry[1].offset

← each entry's offset/size locate its dep section

Bundle TOC — at @192, immediately after the header

```
[ entry_count : u16 ] then BundleEntry × entry_count
```

`BundleEntry` — 152 bytes, no padding

`name[64]`	module name, NUL-term, no ".flx"	`sha256[32]`	dep's whole-file SHA-256 — the `library()` pin
`version : u32`	0xMMmmppbb	`sig_alg : u8`	dep's signature scheme (0 = unsigned)
`offset : u64`	absolute offset of dep's main section	`pubkey[32]`	dep's Ed25519 key (zero if unsigned)
`size : u32`	byte length of that section	`reserved[7]`	zero

A bundled dep keeps only its main section (its own header is dropped), so the pin travels in `sha256` here — a bundled and an on-disk dep pin identically.
The region ranges (`offset`, `size`) are bounds-checked against the file before any allocation.

7 · Integrity, byte order & load-time verification

- **Byte order:** little-endian throughout — header, string pool, bytecode operands and JIT IR (native order on x86/ARM, no byte-swap to decode).
- **Pin / fingerprint:** whole-file SHA-256 equals `sha256sum file.flx`; `library(name, version, hash)` enforces it.
- **Signature:** Ed25519 over the domain-separated digest above; trust anchors are compiled in, plus the user's `~/.flaris/trusted_keys`.
- **Verification:** the VM checks operand bounds, jump-target alignment and per-frame stack depth before executing; a bad instruction raises `IllegalInstruction` (16).
- **Portability:** only fixed-width integer types on disk; no pointer-sized fields. Opcode numbering is versioned — an older chunk is rejected with a recompile hint (version floor).

‡ the 32-byte `name` (output basename) is part of the signed / fingerprinted bytes, so renaming the output changes the pin.